

# CS2308算法与复杂性 课程笔记

uu-matter543

2024-2025-2

## 目录

|          |                         |           |
|----------|-------------------------|-----------|
| <b>1</b> | <b>引言</b>               | <b>6</b>  |
| <b>2</b> | <b>数学归纳法</b>            | <b>7</b>  |
| 2.1      | 引言                      | 7         |
| 2.2      | 平面区域计数问题                | 7         |
| 2.3      | 简单着色问题                  | 8         |
| 2.4      | 复杂加法问题                  | 8         |
| 2.5      | 简单的不等式                  | 9         |
| 2.6      | 欧拉公式问题                  | 9         |
| 2.7      | 独立集与顶点的覆盖性              | 10        |
| 2.8      | 无重边的路                   | 10        |
| 2.9      | 均值不等式                   | 11        |
| 2.10     | 循环不变量                   | 11        |
| <b>3</b> | <b>算法分析</b>             | <b>13</b> |
| 3.1      | 引言                      | 13        |
| 3.2      | $O, \Omega, \Theta$ 表示法 | 13        |
| 3.3      | 算法的时间                   | 14        |
| 3.4      | 算法分析中的常用函数及其渐进性质        | 16        |
| 3.4.1    | 对数函数                    | 16        |
| 3.4.2    | 顶函数和底函数                 | 16        |
| 3.4.3    | 阶乘                      | 16        |
| 3.4.4    | 二项式系数                   | 16        |
| 3.4.5    | 鸽巢原理                    | 16        |
| 3.5      | 和式                      | 17        |

|          |                  |           |
|----------|------------------|-----------|
| 3.5.1    | 渐进性质             | 17        |
| 3.5.2    | 积分近似             | 18        |
| 3.6      | 递推关系             | 18        |
| 3.6.1    | 合理猜测             | 18        |
| 3.6.2    | 分治关系             | 19        |
| 3.6.3    | 涉及全部历史的递推关系      | 20        |
| <b>4</b> | <b>基于归纳的算法设计</b> | <b>22</b> |
| 4.1      | 引言               | 22        |
| 4.2      | 多项式求和            | 22        |
| 4.3      | 最大导出子图           | 22        |
| 4.4      | 一对一映射            | 23        |
| 4.5      | 社会名流问题           | 23        |
| 4.6      | 轮廓线问题            | 23        |
| 4.7      | 计算平衡因子           | 24        |
| 4.8      | 最大连续子列和          | 24        |
| 4.9      | 动态规划: 背包问题       | 25        |
| 4.10     | 总结               | 25        |
| <b>5</b> | <b>序列与集合算法</b>   | <b>26</b> |
| 5.1      | 引言               | 26        |
| 5.2      | 二分搜索             | 26        |
| 5.2.1    | 经典二分搜索           | 26        |
| 5.2.2    | 循环序列的二分搜索        | 26        |
| 5.2.3    | 二分搜索特殊下标         | 26        |
| 5.2.4    | 未知长度序列的二分搜索      | 26        |
| 5.2.5    | 重叠子序列问题          | 27        |
| 5.2.6    | 方程求解             | 27        |
| 5.3      | 内插搜索             | 27        |
| 5.4      | 排序               | 27        |
| 5.4.1    | 基于数值的排序          | 27        |
| 5.4.2    | 基于比较的排序          | 28        |
| 5.5      | 顺序统计问题           | 28        |
| 5.5.1    | 最大数最小数问题         | 28        |
| 5.5.2    | 第 $k$ 小数问题       | 29        |
| 5.6      | 串匹配              | 29        |

|          |                 |           |
|----------|-----------------|-----------|
| 5.6.1    | BF算法            | 30        |
| 5.6.2    | KMP算法           | 30        |
| 5.6.3    | BM算法            | 31        |
| 5.7      | 中位数问题           | 31        |
| 5.7.1    | 加权中位数           | 31        |
| 5.7.2    | 等长有序数组的中位数      | 32        |
| 5.8      | 回文串             | 32        |
| 5.9      | 序列修正问题          | 33        |
| 5.10     | 频繁元素问题          | 33        |
| 5.10.1   | 查找众数            | 33        |
| 5.10.2   | 频繁元素            | 33        |
| 5.11     | 最长单调序列          | 34        |
| 5.12     | 查找前两大元素         | 34        |
| 5.13     | 求多重集合的模         | 35        |
| <b>6</b> | <b>图算法</b>      | <b>36</b> |
| 6.1      | 图的基本概念 (先修课程知识) | 36        |
| 6.2      | 图的遍历            | 36        |
| 6.2.1    | 深度优先搜索 (DFS)    | 36        |
| 6.2.2    | 广度优先搜索 (BFS)    | 36        |
| 6.3      | 拓扑排序            | 37        |
| 6.4      | 最短路径问题          | 37        |
| 6.4.1    | 无权有向图的单源最短路径    | 37        |
| 6.4.2    | 正权有向图的单源最短路径    | 37        |
| 6.4.3    | 任意权有向图的单源最短路径   | 38        |
| 6.4.4    | 任意图的全部最短路径      | 38        |
| 6.4.5    | 最短路径算法应用: 负环问题  | 38        |
| 6.5      | 传递闭包问题          | 38        |
| 6.6      | 最小生成树           | 39        |
| 6.7      | 图的分解            | 39        |
| 6.7.1    | 双连通分支           | 39        |
| 6.7.2    | 强连通分支           | 40        |
| 6.7.3    | 图分解的应用          | 41        |
| 6.8      | 匹配问题            | 41        |
| 6.8.1    | 稠密图匹配           | 41        |
| 6.8.2    | 偶图匹配            | 42        |

|          |                |           |
|----------|----------------|-----------|
| 6.9      | 非常稠密图的哈密顿回路    | 42        |
| <b>7</b> | <b>几何算法</b>    | <b>43</b> |
| 7.1      | 基本概念           | 43        |
| 7.2      | 点与多边形          | 43        |
| 7.3      | 无序点构造多边形       | 43        |
| 7.4      | 凸包问题           | 43        |
| 7.5      | 最近点对问题         | 44        |
| 7.6      | 线段交点问题         | 45        |
| 7.6.1    | 水平垂直线段的交点      | 45        |
| 7.6.2    | 任意线段交点         | 45        |
| 7.7      | 平面图问题          | 46        |
| 7.7.1    | 基本概念和性质        | 46        |
| 7.7.2    | 半平面问题          | 46        |
| 7.7.3    | 美术馆问题          | 47        |
| 7.7.4    | 单调多边形的三角划分     | 47        |
| 7.7.5    | 一般多边形划分为单调多边形  | 48        |
| 7.7.6    | Voronoi图       | 49        |
| <b>8</b> | <b>代数与数值算法</b> | <b>51</b> |
| 8.1      | 引言             | 51        |
| 8.2      | 求幂运算           | 51        |
| 8.3      | 最大公约数          | 51        |
| 8.4      | 多项式乘积          | 51        |
| 8.5      | 方阵乘法           | 52        |
| 8.6      | 布尔阵乘法          | 52        |
| 8.7      | 矩阵连乘           | 53        |
| 8.8      | 快速傅里叶变换FFT     | 53        |
| 8.9      | (*)非线性方程组数值解   | 54        |
| <b>9</b> | <b>归约</b>      | <b>55</b> |
| 9.1      | 引言             | 55        |
| 9.2      | 循环平移问题         | 55        |
| 9.3      | 代表集问题          | 55        |
| 9.4      | 序列比较问题         | 55        |
| 9.5      | 无向图的完全子图       | 55        |
| 9.6      | 线性规划           | 56        |

|           |                         |           |
|-----------|-------------------------|-----------|
| 9.6.1     | 网络流量归约 . . . . .        | 56        |
| 9.6.2     | 静态路由归约 . . . . .        | 56        |
| 9.7       | 下界的归约 . . . . .         | 57        |
| 9.7.1     | 问题的下界 . . . . .         | 57        |
| 9.7.2     | 决策树模型 . . . . .         | 57        |
| 9.7.3     | 简单多边形问题复杂度 . . . . .    | 57        |
| 9.7.4     | 矩阵相乘下界归约 . . . . .      | 57        |
| <b>10</b> | <b>P=NP问题</b>           | <b>59</b> |
| 10.1      | 引言 . . . . .            | 59        |
| 10.2      | 多项式归约 P=NP问题 . . . . .  | 59        |
| 10.3      | NP完全问题的归约 . . . . .     | 59        |
| 10.3.1    | 顶点覆盖问题 . . . . .        | 60        |
| 10.3.2    | 支配集问题 . . . . .         | 60        |
| 10.3.3    | 3-SAT问题 . . . . .       | 61        |
| 10.3.4    | 团集问题 . . . . .          | 62        |
| 10.3.5    | 3着色问题 . . . . .         | 62        |
| 10.3.6    | 总结 . . . . .            | 63        |
| 10.4      | (*) NP完全问题的处理 . . . . . | 63        |
| 10.4.1    | 回溯法 . . . . .           | 63        |
| 10.4.2    | 分支限界法 . . . . .         | 64        |
| 10.4.3    | 近似算法 . . . . .          | 65        |
| 10.4.4    | 随机算法 . . . . .          | 65        |
| <b>11</b> | <b>(*) 并行算法</b>         | <b>66</b> |
| <b>A</b>  | <b>第6章扩展：机器学习基础</b>     | <b>67</b> |
| <b>B</b>  | <b>第9章扩展：线性规划的求解</b>    | <b>68</b> |

# 1 引言

算法有如下基本性质：

- 有0或多个输入
- 至少有1个输出
- 每个步骤必须清晰明确
- 在合理的、有限的步骤内完成

计算机问题求解的步骤：

1. 问题理解与分析
2. 算法设计
3. 算法正确性分析，若不正确，则要重新设计算法
4. 算法复杂性分析，若不合理，则要重新设计算法
5. 编程实现算法

算法的描述：伪代码，兼具形象性与准确性

## 2 数学归纳法

### 2.1 引言

**基本原理：**对命题  $P(n), \forall n \in \mathbb{N}_+$ ，若有

- 对  $n = 1$  成立
- 根据对  $n = k$  成立，可推出对  $n = k + 1$  也成立

则命题  $P(n)$  对  $\forall n \in \mathbb{N}_+$  成立

**例1** 对  $x \in \mathbb{N}_+ (x \neq 1)$  和  $n \in \mathbb{N}_+$ ，证明  $x^n - 1$  能被  $x - 1$  整除

**解** 对  $x$  进行归纳较复杂，因此对  $n$  进行归纳

$n = 1$  时  $\frac{x-1}{x-1} = 1$  成立

设该命题对  $n = k$  成立，即  $\frac{x^k - 1}{x - 1} = A \in \mathbb{N}_+$ ，则  $n = k + 1$  时

$$\frac{x^{k+1} - 1}{x - 1} = \frac{x^{k+1} - x^k + x^k - 1}{x - 1} = x^k + \frac{x^k - 1}{x - 1} = x^k + A \in \mathbb{N}_+$$

由数学归纳法知该命题对  $n \in \mathbb{N}_+$  成立

**启示** 归纳变量的选择对归纳法的使用有重要影响

**变式1（强归纳）：**对命题  $P(n), \forall n \in \mathbb{N}_+$ ，若有

- 对  $n = 1$  成立
- 根据对  $n \leq k$  成立，可推出对  $n = k + 1$  成立

则命题  $P(n)$  对  $\forall n \in \mathbb{N}_+$  成立

**变式2（分别归纳）：**对命题  $P(n), \forall n \in \mathbb{N}_+$ ，若可以分别归纳证明

- 对  $n = 2k - 1, k \in \mathbb{N}_+$  成立
- 对  $n = 2k, k \in \mathbb{N}_+$  成立

则命题  $P(n)$  对  $\forall n \in \mathbb{N}_+$  成立

**变式3（部分归纳）：**对命题  $P(n), n \in \mathbb{N}_+$ ，若

- 对  $n = 1$  成立
- 由对  $n = 2^k$  成立，可推出对  $n = 2^{k+1}$  成立

则命题  $P(n)$  对  $\forall n = 2^k, k \in \mathbb{N}$  成立

### 2.2 平面区域计数问题

**例2** 平面上  $n$  条直线（不存在平行直线，也不存在  $\geq 3$  条直线交于一点）可将平面划分为多少块区域？

**解**  $n = 1, R = 2; n = 2, R = 4; n = 3, R = 7; n = 4, R = 11; \dots$

构造结论  $R = 1 + \frac{n(n+1)}{2}$ ，其含义可解释为：开始为 1 个区域，第  $n$  条直线能增加  $n$  个区域

证明：对  $n$  进行归纳， $n = 1$  时显然成立

设该命题对  $n - 1$  成立，则第  $n$  条直线与前面的  $n - 1$  条直线均相交，产生  $n - 1$  个交点，即第  $n$  条直线被分成  $n$  段，每一段分别将  $n - 1$  时的 1 个区域分为 2 个区域，即增加了  $n$  个区域

由数学归纳法知结论成立

**启示** 由归纳假设推导时关注量的增长率而非量本身；合理猜测并证明的方法

## 2.3 简单着色问题

**例3** 证明：平面上任意条直线划分的区域可以仅用 2 种颜色着色，且任意有相邻边的 2 个区域不同色

**解**  $n = 1$  显然成立；设  $n - 1$  时命题成立

添加第  $n$  条直线后，一侧区域颜色不变，另一侧区域颜色反转

正确性证明：设添加直线  $l_n$  后区域  $R_1, R_2$  相邻

- 若  $R_1, R_2$  在  $l_n$  同侧，则其颜色要么都不变，要么都反转因为  $R_1, R_2$  在  $l_n$  同侧，所以加入  $l_n$  之前  $R_1, R_2$  属于不同区域，加入  $l_n$  之后颜色仍不同，成立
  - 若  $R_1, R_2$  在  $l_n$  异侧，则其中一个区域颜色不变，另一个区域颜色反转因为  $R_1, R_2$  在  $l_n$  异侧，所以加入  $l_n$  之前  $R_1, R_2$  属于相同区域，加入  $l_n$  之后颜色不同，成立
- 因此添加直线  $l_n$  后仍有可行着色方案，由数学归纳法知结论成立

**启示** 追求灵活性，通过算法设计证明方案可行

## 2.4 复杂加法问题

**例4** 考察下面的三角形，求三角形中第  $i$  行的元素之和

|    |    |     |   |     |    |
|----|----|-----|---|-----|----|
|    |    |     | 1 |     |    |
|    |    | 3   |   | 5   |    |
|    | 7  |     | 9 |     | 11 |
| 13 |    | 15  |   | 17  | 19 |
| 21 | 23 | ... |   | ... |    |



**解**  $n = 1, \text{sum} = 1; n = 2, \text{sum} = 8; n = 3, \text{sum} = 27; n = 4, \text{sum} = 64; \dots$  因此构造结论  $\text{sum} = n^3$

$n = 1$ , 第 1 行的和为 1, 成立; 设  $n$  成立, 即第  $n$  行的和为  $n^3$ , 则第  $n + 1$  行的前  $n$  个数分别比第  $n$  行前  $n$  个数大  $2n$ , 所以前  $n$  个数的和为  $n^3 + 2n^2$

而根据归纳假设, 第  $n + 1$  行的和应为  $(n + 1)^3$ , 所以第  $n + 1$  行最后的数应为  $(n + 1)^3 - n^3 - 2n^2 = n^2 + 3n + 1 = (n + 1)^2 + n$ , 即第  $n$  行最后的数应为  $n^2 + n - 1$

$n = 1$ , 第 1 行最后的数为  $1^2 + 1 - 1 = 1$ , 成立; 设第  $n$  行最后的数为  $n^2 + n - 1$ , 则第  $n + 1$  行有  $n + 1$  个数, 相邻元素相差 2, 最后的数应为  $n^2 + n - 1 + 2(n + 1) = n^2 + 3n + 1 = (n + 1)^2 + n$ , 成立

由数学归纳法知结论成立

**启示** 多层归纳, 逐步简化, 可实现复杂问题的归纳证明

## 2.5 简单的不等式

**例5** 证明  $\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots + \frac{1}{2^n} < 1$

**解**  $n = 1, \frac{1}{2} < 1$ , 成立; 设  $\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots + \frac{1}{2^n} < 1$  成立, 则

$$\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots + \frac{1}{2^{n+1}} = \frac{1}{2} + \frac{1}{2} \left( \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots + \frac{1}{2^n} \right) < \frac{1}{2} + \frac{1}{2} = 1$$

由数学归纳法知结论成立

**启示** 注意归纳假设条件的应用

## 2.6 欧拉公式问题

**例6** 证明: 任意线段构成的平面连通图的结点数  $V$ , 边数  $E$ , 区域数  $F$  满足  $V + F = E + 2$

**解** 嵌套归纳, 外层对  $F$  归纳, 内层对  $V$  归纳

1. 先证  $V + F = E + 2$  对  $F = 1$  成立, 即  $V + 1 = E + 2$ , 此时图为树

- $V = 1$  时,  $E = 0$ , 有  $V + 1 = E + 2$
- 设  $V = n$  时有  $V + 1 = E + 2$ , 即  $E = n - 1$ , 则  $V = n + 1$  时, 由于图为树, 必存在结点  $v$  只与 1 条边相关联, 去掉  $v$  时即为  $V = n$  的情况; 加上  $v$  时, 为保证联通且仍为树, 只能让  $v$  与  $V = n$  的树中的 1 条边相关联, 因此成立
- 由数学归纳法知  $V + F = E + 2$  对  $F = 1$  成立

2. 再证  $V + F = E + 2$  若对  $F = n$  成立, 则 (不失一般性可设  $V$  不变) 对  $F = n + 1$  成立

- $F = n + 1$  时, 至少存在 1 个区域由回路围成, 去掉该回路的 1 条边, 即为  $F = n$  的情况; 加上上述边,  $E$  和  $F$  均增加 1, 成立

由数学归纳法知  $V + F = E + 2$  对任意  $V, E, F$  成立

启示 对多个参数可以运用嵌套的方式归纳证明

## 2.7 独立集与顶点的覆盖性

例7  $G = (V, E)$  为有向图,  $S(G)$  为独立集 (任意 2 个点之间都不存在边)

证明: 存在  $S(G)$ , 使得  $V(G)$  中所有结点都可从  $S(G)$  中的某个结点出发, 通过长度不超过 2 的道路到达

解  $|V| = 1$  显然成立

设  $|V| < n$  时成立, 当  $|V| = n$  时, 对于  $V$  中的某个结点  $v$ , 构造

$$N(v) = \{v\} \cup \{w \in V | (v, w) \in E\}$$

进行操作  $V - N(v)$  并删去相关联的边, 可由  $G$  导出图  $H$ , 且满足  $V(H) < n$ , 由归纳假设, 存在  $S(H)$  满足上述题设

1. 若  $S(H) \cup \{v\}$  也是独立集, 则  $N(v) - \{v\}$  中的结点由  $v$  开始 1 步可达, 成立
2. 若  $S(H) \cup \{v\}$  不是独立集, 则  $\exists w \in S(H), (w, v) \in E$ , 对于  $S(H)$ ,  $N(v)$  中的结点由  $w$  开始 2 步可达, 成立

由数学归纳法知结论成立

启示 运用强归纳法, 从  $< n$  的情况推知  $= n$  的情况成立

## 2.8 无重边的路

例8  $G = (V, E)$  为连通无向图,  $O \in V$  为  $G$  中度数为奇数的结点的集合

证明: 可以把  $O$  中的结点分成结点对, 并且可为每对结点之间找到路径, 使得每条路径之间没有重复的边

解 对边数归纳, 只有 1 条边, 显然成立

设  $< n$  条边时成立, 当  $= n$  条边时, 若  $O$  非空, 任取两点  $v_1, v_2$ , 去掉之间的路径, 则为  $< n$  条边的情况

此时并不能保证图联通, 但若问题改为 “ $G = (V, E)$  为任意无向图”, 则结论成立; 而连通无向图是任意无向图的子集, 因此题设也成立

启示 有时扩大问题范围更易于归纳

## 2.9 均值不等式

例9 证明均值不等式

$$(x_1 x_2 x_3 \cdots x_n)^{\frac{1}{n}} \leq \frac{1}{n}(x_1 + x_2 + x_3 + \cdots + x_n)$$

解 首先证明其对  $n = 2^k, k \in \mathbb{N}$  成立, 然后反向归纳

$k = 0$ ,  $x_1 \leq x_1$ , 成立;  $k = 1$ ,  $\sqrt{x_1 x_2} \leq \frac{x_1 + x_2}{2}$ , 易证; 设  $n = 2^k$  成立, 则  $n = 2^{k+1}$  时

$$\begin{aligned} (x_1 x_2 x_3 \cdots x_{2n})^{\frac{1}{2n}} &= \sqrt{(x_1 x_2 x_3 \cdots x_n)^{\frac{1}{n}} (x_{n+1} x_{n+2} x_{n+3} \cdots x_{2n})^{\frac{1}{n}}} \\ &\leq \frac{(x_1 x_2 x_3 \cdots x_n)^{\frac{1}{n}} + (x_{n+1} x_{n+2} x_{n+3} \cdots x_{2n})^{\frac{1}{n}}}{2} \\ &\leq \frac{(\frac{1}{n}(x_1 + x_2 + x_3 + \cdots + x_n) + \frac{1}{n}(x_{n+1} + x_{n+2} + x_{n+3} + \cdots + x_{2n}))}{2} \\ &= \frac{1}{2n}(x_1 + x_2 + x_3 + \cdots + x_{2n}) \end{aligned}$$

由数学归纳法知均值不等式对  $n = 2^k, k \in \mathbb{N}$  均成立

下面反向归纳, 由  $n$  的情况推知  $n - 1$  的情况

引入  $\delta = \frac{1}{n-1}(x_1 + x_2 + x_3 + \cdots + x_{n-1})$ , 将  $x_n = \delta$  代入  $n$  时的均值不等式

$$(x_1 x_2 x_3 \cdots x_{n-1} \delta)^{\frac{1}{n}} \leq \frac{1}{n}(x_1 + x_2 + x_3 + \cdots + x_{n-1} + \delta) = \delta$$

$$\therefore x_1 x_2 x_3 \cdots x_{n-1} \leq \delta^{n-1} \text{ 即 } (x_1 x_2 x_3 \cdots x_{n-1})^{\frac{1}{n-1}} \leq \frac{1}{n-1}(x_1 + x_2 + x_3 + \cdots + x_{n-1})$$

由反向归纳法可知均值不等式对  $\forall n \in \mathbb{N}_+$  成立

启示 反向归纳法的一般形式: 设  $P(n)$  对  $\mathbb{N}_+$  的任意无穷子集成立, 且由  $P(n)$  对  $n$  成立可推出  $P(n)$  对  $n - 1$  成立, 则有  $P(n)$  对  $\forall n \in \mathbb{N}_+$  成立

## 2.10 循环不变量

定义: 用来证明以循环为主体的算法的正确性, 要点如下:

1. 定义与循环体操作的数据相关的某个定值
2. 求出该定值再循环开始前的值
3. 证明该定值在一轮循环过后保持不变
4. 该定值直到循环结束保持不变说明了算法正确性

**例10** 证明下列十进制转二进制算法的正确性

```

// Algorithm: Convert decimal number into binary number
// Input: n(a demical integer)
// Output: b[](stands for a binary number)
init t = n, k = 0
while (t > 0) begin
    k = k + 1
    b[k] = t % 2
    t = t / 2
endwhile

```

**解** 循环不变量  $t \cdot 2^k + m$ ,  $m$  为  $b[1..k]$  代表的整数, 表示为  $m = b[1] \cdot 2^0 + b[2] \cdot 2^1 + b[3] \cdot 2^2 + \dots + b[k] \cdot 2^{k-1}$

- 初始  $k = 0, m = 0, n = t$
- 循环  $n = t \cdot 2^k + m$ 
  - $t$  为偶数,  $m$  不变,  $t \rightarrow \frac{t}{2}, k \rightarrow k + 1, n$  保持不变
  - $t$  为奇数,  $b[k + 1] = 1, m \rightarrow m + 2^k, k \rightarrow k + 1, t \rightarrow \frac{t - 1}{2}, n$  保持不变
- 结束:  $t = 0, m = n$ , 说明 ‘b[]’ 表示的整数为  $n$

**常见错误**

从  $n$  推出  $n + 1$  的情况时, “+1” 必须是任意的

例如下面的证明显然是荒谬的

求证: 世界上所有马颜色相同

证明:  $n = 1$  显然成立; 设命题对  $n$  成立, 则  $n + 1$  时, 第  $1..n$  匹马颜色相同, 第  $2..n + 1$  匹马颜色相同, 则第  $1..n + 1$  匹马颜色相同, 所以世界上所有马颜色相同

### 3 算法分析

#### 3.1 引言

**算法分析的目的：**预测算法行为，包括正确性分析、时间复杂性分析、空间复杂性，以判断算法是否合理、算法的优劣（一般分析问题规模较大的情况）

**决定算法时间的因素：**

- 问题规模
- 代码的质量（重复的不必要调用会降低性能）
- 计算机系统性能
- 算法的时间复杂性（本课程重点关注的内容）

算法的时间复杂性用渐进时间复杂性表示，即问题规模趋向于无穷大时，算法时间的变化趋势和增长速度

算法的空间复杂性是运行算法所需的临时空间，以时间复杂性为上界

#### 3.2 $O, \Omega, \Theta$ 表示法

- $O$ ：若  $\exists c > 0, N > 0 : \forall n \geq N, g(n) \leq c \cdot f(n)$ ，则记作  $g(n) = O(f(n))$ ，有如下性质：
  1.  $\forall c > 0, a > 1, (f(n))^c = O(a^{f(n)})$ ，即增长速度：对数函数  $<$  多项式函数  $<$  指数函数
  2. 若  $f(n) = O(s(n)), g(n) = O(t(n))$ ，则  $f(n) + g(n) = O(s(n) + t(n))$ ， $f(n)g(n) = O(s(n)t(n))$
- $\Omega$ ：若  $\exists c > 0, N > 0 : \forall n \geq N, g(n) \geq c \cdot f(n)$ ，则记作  $g(n) = \Omega(f(n))$
- $\Theta$ ：若  $\exists c_1 > 0, c_2 > 0, N > 0 : \forall n \geq N, c_1 \cdot f(n) \leq g(n) \leq c_2 \cdot f(n)$ ，则记作  $g(n) = \Theta(f(n))$
- $o$ ：若  $\forall c > 0, \exists N : \forall n \geq N, g(n) \leq c \cdot f(n)$ ，则记作  $g(n) = o(f(n))$ 
  1.  $f(n) = o(g(n)) \Leftrightarrow \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$
  2.  $f(n) = o(g(n)) \Leftrightarrow f(n) = O(g(n)) \wedge g(n) \neq O(f(n))$
- 常用的时间复杂性：（ $\prec$  表示增长速度前者慢于后者）

$$1 \prec \log \log n \prec \log n \prec \sqrt{n} \prec n \prec n \log n \prec n^2 \prec n^3 \prec 2^n \prec n! \prec n^n$$

**例1**  $f(n+1) = \_f(n)$ ，举例说明填  $O, \Omega, \Theta$  的情况

**解**  $f(n) = n, f(n+1) = n+1$ ，可以填  $O, \Omega, \Theta$

$f(n) = n!, f(n+1) = (n+1)! = (n+1)f(n)$ ，只能填  $\Omega$

### 3.3 算法的时间

如果问题  $P$  的复杂度为  $\Omega(f(n))$ ，则能够在  $O(f(n))$  时间内求解问题  $P$  的算法均为最优算法

**例2** 若问题  $P$  的复杂度可以证明为  $\Omega(n^2)$ ，求下列算法中可能的最优算法：

$$T_1(n) = 100n^2 + 200n + 300$$

$$T_2(n) = \frac{1}{2}n^2$$

$$T_3(n) = n + k$$

$$T_4(n) = n^3 + 10n^2 \log n + 50n^2$$

**解**  $T_3$  是错误的算法，因为算法时间复杂度不能小于问题时间复杂度  
 $T_1, T_2$  如果正确则是最优算法， $T_4$  不是最优算法

估计算法时间的方法：

- 计算迭代次数
- 计算基本运算频度（注意选择合适的运算单元）
- 分析递推关系

**例3** 分析如下算法的时间复杂度

```
// Input: n
// Output: Count
init count = 0
while (n >= 1)
    for (j = 1..n)
        count++
    endfor
    n = n/2
endwhile
return count
```

**解** 将count++视为最基本的运算单元，设  $2^{k-1} \leq n \leq 2^k$ ，则运算单元数  $\sum_{j=0}^k \frac{n}{2^j} = \Theta(n)$ ，所以时间复杂度  $\Theta(n)$

**例4** 分析插入排序算法的时间复杂度（比较元素使用二分法）

**解** 虽然比较次数期望降低为  $O(n \log n)$ ，但元素交换次数仍为  $O(n^2)$ ，所以时间复杂度仍为  $O(n^2)$

分析算法时，一般以最坏情况和平均情况作为时间复杂度的分析对象，一般以最坏情况作为空间复杂度的分析对象

**输入规模：**输入所占的空间，如

- 排序问题中的数组规模  $n$
- 矩阵运算中的矩阵元素个数  $n^2$
- 以输入数据的大小作为输入规模时，由于机内用二进制表示，所以输入规模为  $\log n$

**XX时间算法：**算法时间复杂性相对于输入规模的函数，如

- 二分查找  $\log n$  相对于输入规模  $n$  为对数时间
- 矩阵加法  $n^2$  相对于输入规模  $n^2$  为线性时间

**例5** 分析素数判断算法的时间

```
// Algorithm: Prime judgement
// Input: n
// Output: false/true
for i = 2 to sqrt(n)
    if (n % i == 0) return false
endfor
return true
```

**解** 输入规模  $O(\log n)$ ，时间复杂度  $O(\sqrt{n}) = O(2^{\frac{\log n}{2}})$ ，所以该算法为指数时间算法

### 3.4 算法分析中的常用函数及其渐进性质

#### 3.4.1 对数函数

$$\log xy = \log x + \log y$$

$$\log x^y = y \log x$$

$$\log_a b = (\log_b a)^{-1}$$

$$\log_x y = \frac{\log_a y}{\log_a x}$$

$$a^{\log_a x} = x$$

$$x^{\log y} = y^{\log x}$$

$$\sum_{i=1}^n [\log i] = \Theta(n \log n)$$

#### 3.4.2 顶函数和底函数

- 底函数:  $\lfloor x \rfloor$  表示不大于  $x$  的最大整数
- 顶函数:  $\lceil x \rceil$  表示不小于  $x$  的最小整数
- $x \in \mathbb{N}$ ,  $\lceil \frac{x}{2} \rceil + \lfloor \frac{x}{2} \rfloor = x$

#### 3.4.3 阶乘

$$n! = \Theta(\sqrt{n} \cdot n^n)$$

$$\log(n!) = \Theta(n \log n)$$

#### 3.4.4 二项式系数

$$\binom{n}{k} = \binom{n}{n-k} = \frac{n!}{k!(n-k)!}$$

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

#### 3.4.5 鸽巢原理

将  $n$  个球放入  $m$  个盒子中, 则

- 存在一个盒子至少装了  $\lceil \frac{n}{m} \rceil$  个球
- 存在一个盒子至多装了  $\lfloor \frac{n}{m} \rfloor$  个球



**例6** 优化以下算法，使其为有限循环

```
// Input: a, b (a < b, and both are positive integers)
// Output: a/b (0.xxx...)
Output("0.")
a = a*10
while (a > 0)
    Output(a/b)
    a = (a%b)/10
endwhile
```

**解** 由于  $a\%b$  最多有  $b$  种取值，则由鸽巢原理， $b$  次循环内必能找到循环体，添加循环次数不大于  $b$  的限制即可

## 3.5 和式

### 3.5.1 渐进性质

$$\begin{aligned}\sum_{j=1}^n j &= \frac{n(n+1)}{2} = \Theta(n^2) \\ \sum_{j=1}^n j^2 &= \frac{n(n+1)(n+2)}{6} = \Theta(n^3) \\ \sum_{j=1}^n c^j &= \frac{c^{n+1} - 1}{c - 1} = \Theta(c^n) \\ \sum_{j=1}^n jc^j &= \frac{nc^{n+2} - nc^{n+1} - c^{n+1} + c}{(c-1)^2} = \Theta(nc^n) \\ \sum_{j=1}^n \frac{1}{j} &= \Theta(\log n) \\ \sum_{j=1}^n \frac{1}{c^j} &= \Theta(1) \\ \sum_{j=1}^n \frac{j}{c^j} &= \Theta(1)\end{aligned}$$

其中  $c$  为常数且  $c > 1$

### 3.5.2 积分近似

对于单调递增函数  $f(n)$ , 有

$$\int_{m-1}^n f(x)dx \leq \sum_{j=m}^n f(j) \leq \int_m^{n+1} f(x)dx$$

**例7** 证明  $\log(n!) = \Theta(n \log n)$

**解**  $\log(n!) = \sum_{i=1}^n \log i$

$$\begin{aligned} \sum_{i=1}^n \log i &= \log n + \sum_{i=1}^{n-1} \log i \\ &\leq \log n + \int_1^n \log x dx \\ &= \log n + n \log n + n \log e + \log e \\ &= O(n \log n) \end{aligned}$$

$$\begin{aligned} \sum_{i=1}^n \log i &= \log 1 + \sum_{i=2}^n \log i \\ &\geq 0 + \int_1^n \log x dx \\ &= n \log n + n \log e + \log e \\ &= \Omega(n \log n) \end{aligned}$$

$$\therefore \log(n!) = \Theta(n \log n)$$

## 3.6 递推关系

所谓递推关系, 即函数的定义体中包含函数本身

### 3.6.1 合理猜测

**例8**  $T(2n) \leq 2T(n) + 2n - 1$ ,  $T(2) = 1$ , 求  $T(n)$  的渐进复杂度

**解** 不妨令  $n = 2^k$

1. 设  $T(n) \leq c \cdot n^2$ ,  $T(2) = 1 \leq 4c$  成立,  $T(2n) \leq 2T(n) + 2n - 1 \leq 2cn^2 + 2n - 1 \leq c \cdot (2n)^2$  成立,  $\therefore T(n) = O(n^2)$
2. 设  $T(n) \leq c \cdot n$ ,  $T(2) = 1 \leq c$  成立,  $T(2n) \leq 2T(n) + 2n - 1 \leq 2cn + 4n - 1 \leq c \cdot 2n$  不成立,  $\therefore n = o(T(n))$

3. 设  $T(n) \leq c \cdot n \log n$ ,  $T(2) \leq c \cdot 2 \log 2$  成立,  $T(2n) \leq 2T(n) + 2n - 1 \leq 2cn \log n + 2n \log 2 - 1 = 2n \log 2n$  成立,  $\therefore T(n) = O(n \log n)$

结论:  $T(n) = O(n \log n)$

$f(n) = \sum_{i=1}^k a_i f(n-i)$  称为  $f(n)$  的  $k$  阶线性齐次递推式

- $k=1$ ,  $f(n) = af(n-1) = a^2f(n-2) = \dots = a^n f(0)$
- $k=2$ ,  $f(n) = a_1f(n-1) + a_2f(n-2) \Rightarrow x^2 = a_1x + a_2$ , 解特征方程  $x^2 = a_1x + a_2$  得到两个根  $r_1, r_2$ 
  - 若  $r_1 \neq r_2$ , 可设  $f(n) = c_1r_1^n + c_2r_2^n$
  - 若  $r_1 = r_2$ , 可设  $f(n) = c_1r^n + c_2nr^n$

### 3.6.2 分治关系

$T(n) = aT\left(\frac{n}{b}\right) + cn^k$  在  $a \geq 1, b \geq 2$  时的解为:

$$T(n) = \begin{cases} O(n^{\log_b a}) & a > b^k \\ O(n^k \log n) & a = b^k \\ O(n^k) & a < b^k \end{cases}$$

证明如下: 设  $T(1) = c$ ,  $n = b^m$ , 则有

$$\begin{aligned} T(n) &= aT\left(\frac{n}{b}\right) + cn^k \\ &= a^2T\left(\frac{n}{b^2}\right) + ac\left(\frac{n}{b}\right)^k + cn^k \\ &= \dots \\ &= c \sum_{i=0}^m a^{m-i} b^{ik} \\ &= ca^m \sum_{i=0}^m \left(\frac{b^k}{a}\right)^i \end{aligned}$$

当  $a > b^k$  时

$$\sum_{i=0}^{\infty} \left(\frac{b^k}{a}\right)^i = \text{Constant} T(n) = O(a^m) = O(a^{\log_b n}) = O(n^{\log_a b})$$

当  $a = b^k$  时

$$\sum_{i=0}^m \left(\frac{b^k}{a}\right)^i = mT(n) = O(m \cdot a^m) = O(n^k \log n)$$

当  $a < b^k$  时, 令  $F = \frac{b^k}{a}$  则

$$T(n) = c \cdot a^m \frac{F^{m+1} - 1}{F - 1} = O((aF)^m) = O(b^{km}) = O(n^k)$$

**主定理:** 若  $T(n) = aT(\frac{n}{c}) + f(n)$ , 则

$$T(n) = \begin{cases} \Theta(n^{\log_c a}) & \exists \epsilon > 0 : f(n) = O(n^{\log_c a - \epsilon}) \\ \Theta(n^{\log_c a} \cdot (\log n)^{k+1}) & \exists k > 0 : f(n) = \Theta(n^{\log_c a} \cdot (\log n)^k) \\ \Theta(f(n)) & \exists \epsilon > 0 : f(n) = \Omega(n^{\log_c a + \epsilon}) \wedge \exists k < 1 : af(\frac{n}{c}) \leq kf(n) \end{cases}$$

注意: 主定理不能完全覆盖所有情况

**例9** 求  $T(n)$  的复杂度

$$(1) T(n) = 5T(\frac{n}{2}) + n^2$$

$$(2) T(n) = 27T(\frac{n}{3}) + n^3 \log n$$

$$(3) T(n) = 5T(\frac{n}{2}) + n^3$$

$$(4) T(n) = 2T(\frac{n}{2}) + \frac{n}{\log n}$$

**解** 套用主定理即可

$$(1) f(n) = n^2 \Rightarrow \exists \epsilon > 0 : f(n) = O(n^{\log_2 5 - \epsilon}) \therefore T(n) = \Theta(n^{\log_2 5})$$

$$(2) f(n) = n^3 \log n \Rightarrow \exists k = 1 > 0 : f(n) = \Theta(n^{\log_3 27} \cdot (\log n)^k) \therefore T(n) = \Theta(n^3 \log^2 n)$$

$$(3) f(n) = n^3 \Rightarrow \exists \epsilon > 0 : f(n) = O(n^{\log_2 5 + \epsilon})$$

$$af(\frac{n}{c}) = \frac{5}{8}n^3 \Rightarrow \exists k = \frac{5}{8} < 1 : af(\frac{n}{c}) \leq kf(n) \therefore T(n) = \Theta(n^3)$$

$$(4) f(n) = \frac{n}{\log n}, n^{\log_c a} = n, f(n) = o(n) \text{ 不满足主定理的任意条件, 不可推断}$$

### 3.6.3 涉及全部历史的递推关系

涉及全部历史的递推关系, 即函数依赖于先前所有的函数值, 可用历史消除法加以处理

**例10** 求以下  $T(n)$  的复杂度

$$(1) T(n) = c + \sum_{i=1}^{n-1} T(i), n \geq 2$$

$$(2) T(n) = n - 1 + \frac{2}{n} \sum_{i=1}^{n-1} T(i), n \geq 2$$

解 历史消除法

$$(1) T(2) = c + T(1) = O(1)$$

$$T(n-1) = c + \sum_{i=1}^{n-2} T(i)$$

$$T(n) - T(n-1) = T(n-1)$$

$$T(n) = 2T(n-1) = \cdots = 2^{n-2}T(2) = O(2^n)$$

$$(2) T(n+1) = n + \frac{2}{n+1} \sum_{i=1}^n T(i)$$

$$\begin{aligned} T(n+1) &= \frac{n+2}{n+1}T(n) + \frac{2n}{n+1} \\ &\leq \frac{n+2}{n+1}T(n) + 2 \\ &\leq \frac{n+2}{n+1}(\frac{n+1}{n}T(n) + 2) + 2 \\ &\leq \cdots \\ &\leq 2n(\frac{1}{n} + \frac{1}{n-1} + \cdots + \frac{1}{3}) \end{aligned}$$

$$\therefore T(n) = O(n \log n)$$

## 4 基于归纳的算法设计

### 4.1 引言

- 基本思想：从简单到复杂
- 途径：
  - 横向分解：将原问题分解为互不相同的子问题（模块化）
  - 纵向分解：将原问题分解为与原问题结构相似但规模更小的子问题，然后将子问题的结果归并（递归）
- 原理：解决小规模问题是可能的，且每个问题的解答都可由更小规模问题的解答构造出来
  - 归纳：问题规模由  $n-1$  到  $n$
  - 分治：问题规模由  $\frac{n}{k}$  到  $n$
  - 动态规划：问题规模由  $< n$  到  $n$
  - 增强归纳：由条件更多的问题求解原问题

### 4.2 多项式求和

问题：给定  $x$  的值，求  $P_n(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$  的值，将乘法视为基本的运算单元

- 直接算法： $P_n(x) = P_{n-1}(x) + a_n x^n$   
 $T(n) = T(n-1) + n$ ,  $T(n) = \frac{n(n+1)}{2} = \Theta(n^2)$
- 改进（增强归纳假设）：改为问题  $Q_n(x)$ ，既求  $P_n(x)$ ，也求  $x^n$   
 $Q_{n-1}(x) \rightarrow Q_n(x) : x^{n-1} \rightarrow x^n$ ,  $P_{n-1}(x) \rightarrow P_n(x)$  需要 2 次乘法  
 $T(n) = T(n-1) + 2$ ,  $T(n) = \Theta(n)$
- 进一步改进（改变归纳方向）：改为问题  $R_n(x)$   
 其中  $R_0(x) = a_n$ ,  $R_i(x) = x R_{i-1}(x) + a_{n-i}$ ，每次迭代只需要 1 次乘法  
 $T(n) = T(n-1) + 1$ ,  $T(n) = \Theta(n)$

### 4.3 最大导出子图

问题：给定无向图  $G = (V, E)$  及整数  $k$ ，找到  $G$  中规模最大的导出子图  $H = (U, F)$ ， $H$  中所有顶点的度数  $\geq k$ ，或证明不存在  $H$

- 算法设计：删除所有度数  $< k$  的顶点，并一并删除新产生的度数  $< k$  的顶点，而无需考虑删除的次序
- 算法正确性：设算法对  $n-1$  个结点的图成立，则对于  $n$  个结点的图
  - $n \leq k$  所求导出子图不存在，算法成立

- $n = k + 1$  图为完全图时为所求导出子图，否则所求导出子图不存在，算法成立
- $n > k + 1$  时
  - \* 若所有结点度数  $\geq k$ ，则该图即为所求导出子图
  - \* 若  $\exists v, d(v) < k$ ，则  $v$  一定会被删除
 删除  $v$  及相关的边后，由归纳假设知算法成立

#### 4.4 一对一映射

**问题：**给定集合  $A$  及函数  $f: A \rightarrow A$ ，寻找元素最多的  $S \subseteq A$ ，满足  $f: S \rightarrow S$  且  $f$  为双射，求  $S$

- 归纳（减小问题规模）：寻找  $A$  上未被其他元素映射到的  $i$ （即  $\forall x \in A, f(x) \neq i$ ），将其删除，可使问题规模由  $n$  减小到  $n - 1$ ，同时将  $i$  删除后产生的未被其他元素映射到的元素也一并删除（可用队列实现）；若不存在这样的  $i$ ，说明此时的集合即为所求的  $S$
- 算法时间复杂度：每个元素最多入队出队各一次，时间复杂度为  $\Theta(n)$

#### 4.5 社会名流问题

**问题：** $n$  个人中，若  $A$  被其余  $n - 1$  个人知道，但不知道其余  $n - 1$  个人，则称  $A$  为社会名流；给定  $n$  个人相识关系的邻接矩阵，求这  $n$  个人中是否存在社会名流

- 归纳（减小问题规模）：最多只有 1 个人是社会名流  
 当询问  $A$  是否知道  $B$  时，根据下述规则，可将问题规模由  $n$  减小到  $n - 1$ 
  - 若  $A$  知道  $B$ ，则  $A$  不可能是社会名流，删除  $A$
  - 若  $A$  不知道  $B$ ，则  $B$  不可能是社会名流，删除  $B$
 最后排除到只剩 1 个人，可以遍历邻接矩阵中对应的 1 行和 1 列以核验其身份
- 算法时间复杂度：每次询问可将问题规模减小 1，最多询问  $n - 1$  次，时间复杂度为  $\Theta(n)$

#### 4.6 轮廓线问题

**问题：**给出  $n$  个建筑的二维轮廓，求城市的轮廓线，其中建筑的ADT为  $\langle L, H, R \rangle$ ， $L$  为左基线， $H$  为高度， $R$  为右基线

- 直接算法：每添加一栋建筑更新一次轮廓线  
 $T(n) = T(n - 1) + \Theta(n) \quad T(n) = \Theta(n^2)$
- 分治算法：将 2 个轮廓线合并依然是  $\Theta(n)$  时间  
 $T(n) = T\left(\frac{n}{2}\right) + \Theta(n) \quad T(n) = \Theta(n \log n)$ 
 分治算法框架：

```

solve_or_divide(P) begin
    if (P.size() <= n_0) solve(P)
    divide(P) // divide P into P_1, P_2, P_3, ..., P_k
    for (i = 1 to k) solve_or_divide(P_i)
    merge(result[1..k])
    return result
end solve_or_divide

```

分治算法流程：

1. 分：将原问题分解为  $k$  个子问题，每个问题规模为  $O(n)$
2. 治：若问题规模大于阈值  $n_0$ ，则进一步分解；否则求解该问题
3. 合：整合  $k$  个子问题的解，形成原问题的解

分治算法复杂度分析：

$$T(n) = \text{div}(n) + \sum_{k=1}^n T(k) + \text{merge}(n, k)$$

## 4.7 计算平衡因子

问题：给定二叉树，计算内部结点和根节点的平衡因子

- 增强归纳假设：由于平衡因子不依赖于子结点的平衡因子，而只依赖于子树的高度，因此增强归纳假设为求出平衡因子，同时给出子树高度，便于求父结点的平衡因子

## 4.8 最大连续子列和

问题： $x[1..n]$  为实序列，求  $x[i..j]$  使得其和为原序列的所有连续子列和中最大

- 归纳：假设已找到  $x[1..n-1]$  的最大连续子列和
  - 若最大和的连续子列为  $x[i..n-1]$ ，只需要考虑  $x_n$  是否大于 0 即可：若  $x_n > 0$ ，则将  $x_n$  加入所求子列，否则不加入
  - 若最大和的连续子列为  $x[i..j]$  ( $j < n-1$ )，需要比较  $x[i..j]$  的和与  $x[1..n-1]$  的最大后缀子列和加上  $x[n]$  的和

因此将归纳假设增强为：既求出最大子列和，也求出最大后缀子列和

维护最大后缀子列和的方法为：若  $x[n]$  加入使得最大后缀子列和变为负数，则将最大后缀子列和重置为 0 即可；否则最大后缀子列和不变

算法时间复杂度  $\Theta(n)$ （只需要扫描序列 1 遍）



## 4.9 动态规划：背包问题

**问题：**背包中有  $n$  个物品，价值分别为  $k[1..n]$ ，求物品组合使得其价值之和恰好为  $k$ ，或说明不存在这样的物品组合

- 直接归纳假设：假设已知  $k[1..n-1]$  中是否存在这样的物品组合  
但若答案为不存在，则归纳假设不能用于求解  $k[1..n]$
- 增强归纳假设：假设可以求解  $k[1..n-1]$  中是否存在这样的物品组合  
 $T(n) = 2T(n-1)$   $T(n) = \Theta(2^n)$  复杂度很高
- 强归纳法（动态规划）：不仅利用  $k[1..n-1]$  的解，而是利用所有小规模问题的解  
已知  $k[1..i-1]$  中是否存在价值为  $[1..k-1]$  的物品组合，记为  $P[i-1, j] (j = 1, 2, 3, \dots, n-1)$   
则当  $k[n]$  加入时有  $P[i, j] = \text{true} \Leftrightarrow P[i-1, j] = \text{true} \vee P[i-1, j-k[i]] = \text{true}$
- 动态规划求解流程：
  1. 设立二维数组  $P[0..n, 0..k]$  保存小规模问题的解
  2. 初始化  $P[0, 0] = \text{true}, P[0, 1..k] = \text{false}$
  3. 然后每次加入  $k[i]$  时，依次求解  $P[i, 0..k]$
  4. 最后取  $P[n, k]$  为问题解
- 由于求解二维数组的每个元素均为  $\Theta(1)$  时间，所以算法的时间复杂度为  $\Theta(nk)$

## 4.10 总结

**常见错误：**

- 不添加递归终止条件导致无限递归
- 从  $n$  扩展到  $n+1$  时忽略  $[n+1]$  元素的一般性

**核心思想：**小规模问题与大规模问题的联系

- 直接方法：去掉某些元素
- 间接方法：分解为多个小规模问题

**分治算法与动态规划的差异：**

- 分治算法：递归思想的应用
- 动态规划：循环迭代思想的应用

## 5 序列与集合算法

### 5.1 引言

序列：考虑元素顺序，元素可重复

集合：不考虑元素顺序，元素不可重复

### 5.2 二分搜索

#### 5.2.1 经典二分搜索

经典二分搜索，即通过待查元素与有序序列中间元素比较，将搜索范围减半，时间复杂度  $O(\log n)$

#### 5.2.2 循环序列的二分搜索

问题：  $x_1, x_2, x_3, \dots, x_n$  中  $\exists x_i : x_i < x_{i+1} < x_{i+2} < \dots < x_n < x_1 < x_2 < x_3 < \dots < x_{i-1}$ ，找出  $x_i$  的位置

- 思想：选择合适的元素进行比较，每次排除范围元素，达到  $O(\log n)$  的复杂度
- 算法：任取  $x_k, x_l (k < l)$ 
  - 若  $x_k < x_l$ ，则  $i \notin (k, l]$
  - 若  $x_k > x_l$ ，则  $i \in (k, l]$

#### 5.2.3 二分搜索特殊下标

问题：整数序列  $a_1, a_2, a_3, \dots, a_n$  递增，求是否存在  $i$  使得  $a_i = i$

- 思想：考查  $a_{n/2}$ ，每次比较可删除一半元素，达到  $O(\log n)$  的复杂度
  - 若  $a_{n/2} = \frac{n}{2}$  则直接得解
  - 若  $a_{n/2} < \frac{n}{2}$  则前半段元素均不满足
  - 若  $a_{n/2} > \frac{n}{2}$  则后半段元素均不满足

#### 5.2.4 未知长度序列的二分搜索

问题：递增序列长度未知，在该序列中查找元素  $z$

- 思想：控制搜索范围实现  $O(\log n)$  的查找
  - 若  $z > x_i$ ，则加倍搜索范围，至多  $\log z$  次
  - 若  $z < x_{2i}$ ，则在  $(i, 2i]$  范围内搜索即可，至多比较  $\log i$  次

### 5.2.5 重叠子序列问题

**问题：**给定序列  $A, B$ ，确定  $i$  的最大值使得  $B^i$  是  $A$  的子序列；其中  $B^i$  表示  $B$  中每个元素重复  $i$  次形成的序列，如  $B = b_1b_2b_3$  则  $B^2 = b_1b_1b_2b_2b_3b_3$

- 思想：控制  $i$  的范围实现  $O(\log k)$  的查找  
 设  $|A| = m, |B| = n$  则  $i < \frac{n}{m}$ ，否则  $B^i$  的长度大于  $A$  的长度  
 可在 1 到  $\frac{n}{m}$  中进行二分搜索，时间复杂度  $O((m+n) \log(m+n))$

### 5.2.6 方程求解

**问题：** $f(a)f(b) < 0, f \in C[a, b]$ ，求  $f(x)$  的零点

- 思想：二分法逼近函数零点  
 若  $f(a + \frac{b-a}{2}) = 0$  则零点就是  $a + \frac{b-a}{2}$   
 否则可将范围缩小至  $(a, a + \frac{b-a}{2})$  或  $(a + \frac{b-a}{2}, b)$   
 $k$  步之后可将零点范围缩小至  $\frac{b-a}{2^k}$

## 5.3 内插搜索

- 思想：根据待检索值在检索区内的相对位置，在一定的邻域内进行搜索
- 适用情况：适用于数据分布均匀的序列
- 优点：可将复杂度由  $O(\log n)$  降低至  $O(\log \log n)$
- 缺点：复杂度即使有所优化也程度有限；内插搜索的运算更加复杂，常系数较大

## 5.4 排序

### 5.4.1 基于数值的排序

- 计数排序：**直接计算小于每个元素的元素个数，直接将元素放在最终位置上，时间复杂度  $O(n)$   
 也可将一定数值范围内的元素视为整体，计算小于该整体的元素数目，将元素置于最终的位置范围上，然后对该范围内的元素进行排序
- 桶排序：**对于取值范围为 1 到  $m$  的  $n$  个元素，可分配  $m$  个桶，根据元素的取值放入桶中，最后遍历桶并收集元素，时间复杂度  $\Theta(m+n)$
- 桶排序扩展：**每个桶中放置多个元素，先对每个桶中的元素进行排序，再遍历桶并收集元素

若桶的个数与元素个数相近，则平均时间复杂度  $\Theta(n)$ ，证明如下：

设  $n_i$  为第  $i$  个桶中的元素个数，则  $n_i \sim B(n, \frac{1}{n})$ ，且对每个桶中的元素排序的时间为  $E(n_i^2)$ （说明：由于平均每个桶中的元素较少，常系数的影响较大，因此对每个桶

中的元素进行排序时，一般不采用  $O(n \log n)$  的排序算法)，则有  $E(n_i) = 1 - \frac{1}{n_i}$ ,  $\therefore E(n_i^2) = D(n_i) + [E(n_i)]^2 = 2 - \frac{1}{n_i} = \Theta(1)$ ，又由于共  $n$  个桶，所以平均时间为  $\Theta(n)$

- **基数排序**：数据位数相同时，可直接按照字典序进行排序

#### 5.4.2 基于比较的排序

- **插入排序**：通过扫描已完成排序的  $n - 1$  个数确定第  $n$  个数应插入的位置，时间复杂度  $O(n^2)$
  - **选择排序**：依次从未排序的部分选择最小元素放在其最终位置，时间复杂度  $O(n^2)$
  - **归并排序**：将序列分为 2 部分，依次递归地归并排序，最后合并为有序序列，时间复杂度  $O(n \log n)$
  - **快速排序**：取任意元素作为主元（一般取中间元素），将序列分为 2 个部分，一部分元素小于主元，另一部分元素大于主元，并递归地进行快速排序，时间复杂度  $O(n \log n)$
  - **堆排序**：建堆，依次入堆并出队，堆操作  $O(\log n)$ ，总时间复杂度  $O(n \log n)$
  - 优化思路：最基本的排序算法按归纳方向分为**插入排序**和**选择排序**
    - 插入排序基础上，将元素与序列的归并，优化为序列与序列的归并，产生**归并排序**；在此基础上强调分治重点，产生**快速排序**
    - 选择排序基础上，优化选择方法，应用历史的比较信息，产生**堆排序**
  - 决策树原理： $n$  个元素的排列数为  $n!$ ，排序结果相当于二叉树的叶子结点，所以二叉树高度至少为  $\log(n!) = \Theta(n \log n)$
- 基于比较的排序实质上是沿着二叉树向下前进，每次比较在结点的左右子结点中选择，直到到达叶子结点，所以基于比较的排序的时间均为  $\Omega(n \log n)$

### 5.5 顺序统计问题

#### 5.5.1 最大数最小数问题

**问题**：求  $x_1, x_2, x_3, \dots, x_n$  的最大数与最小数

- **直接选择**：进行选择排序的单次操作，依次比较并选择最大数与最小数，时间约为  $T(n) = 2n$
- **选择优化**：从  $n - 2$  扩展到  $n$  时，比较  $x_{n-1}$  与  $x_n$ ，之后较大数与前  $n - 2$  个数中的最大数比较，较小数与前  $n - 2$  个数中最小数比较  
递推复杂度  $T(n) = T(n - 2) + 3$   $T(2) = 1$   $T(3) = 3$   $\therefore T(n) = \frac{3n}{2}$ ，较直接选择优化了约 25%
- **分治方法**：分为  $\frac{n}{2}$  的 2 部分，分别递归处理得到  $M_1, m_1, M_2, m_2$ ，之后只要比较  $M_1$  和  $M_2$  以及  $m_1$  和  $m_2$  即可

$$\text{递推复杂度 } T(n) = 2T\left(\frac{n}{2}\right) + 2 \quad \therefore T(n) \approx \frac{3n}{2}$$

### 5.5.2 第 $k$ 小数问题

问题：求  $x_1, x_2, x_3, \dots, x_n$  的第  $k$  小数

- 直接方法：求  $k$  次最小数，时间复杂度  $\Theta(kn)$
- 快排优化：选择主元将集合分为 2 部分，比较主元下标  $index$  和  $k$  的相对位置
  - 若主元位置  $index$  恰好为  $k$  则直接得解
  - 若主元位置  $index$  小于  $k$  则  $k \leftarrow k - index$  之后在后一半搜索
  - 若主元位置  $index$  大于  $k$  则在前半一半搜索

时间复杂性  $\Theta(n)$ ，证明如下：

$$\begin{aligned} T(n) &= (n-1) + \frac{1}{n} \left[ \sum_{i=1}^{k-1} T(n-i) + \sum_{i=k+1}^n T(i-1) \right] \\ &= (n-1) + \frac{1}{n} \left[ \sum_{i=n-k+1}^{n-1} T(i) + \sum_{i=k}^{k-1} T(i) \right] \\ &\leq (n-1) + \frac{1}{n} \max_k \left[ \sum_{i=n-k+1}^{n-1} T(i) + \sum_{i=k}^{k-1} T(i) \right] \end{aligned}$$

取  $k = \left\lceil \frac{n}{2} \right\rceil$  时最大，设  $i < n$  时  $T(i) < 4i$ ，则有

$$\begin{aligned} T(n) &= (n-1) + \frac{1}{n} \left[ \sum_{i=n-\left\lceil \frac{n}{2} \right\rceil+1}^{n-1} 4i + \sum_{i=\left\lceil \frac{n}{2} \right\rceil}^{n-1} 4i \right] \\ &\leq (n-1) + \frac{4}{n} \left[ \sum_{i=\left\lceil \frac{n}{2} \right\rceil}^{n-1} i + \sum_{i=\left\lceil \frac{n}{2} \right\rceil}^{n-1} i \right] \\ &= (n-1) + \frac{8}{n} \sum_{i=\left\lceil \frac{n}{2} \right\rceil}^{n-1} i \\ &\leq (n-1) + \frac{8}{n} \left( \frac{n(n-1)}{2} - \frac{\frac{n}{2}(\frac{n}{2}-1)}{2} \right) \\ &= 4n - 3 < 4n \end{aligned}$$

$$\therefore T(n) = \Theta(n)$$

## 5.6 串匹配

问题：已知字符串  $A, B$ ，求  $B$  在  $A$  中出现的位置

### 5.6.1 BF算法

从  $A$  中出现  $B[1]$  的位置开始匹配，若遇到不能匹配的字符则重新匹配，最坏情况下的时间复杂度  $O(mn)$

### 5.6.2 KMP算法

- 思想：利用匹配过程中的历史信息，设计不回溯算法
- $next[]$  的思想与实现： $next[i]$  为使得  $B[i-j..i-1] = B[1..j]$  最大的  $j$   
构建原理：由  $next[1..i-1]$  强归纳得到  $next[i]$ 
  - 最简单情况  $B[i-1] = B[next[i-1]+1]$ ，此时  $next[i] = next[i-1] + 1$
  - 若  $B[i-1] \neq B[next[i-1]+1]$ 
    - \* 由  $next[]$  的性质可知， $B[i-1]$  之前的  $next[i-1]$  个元素与  $B$  的前  $next[i-1]$  个元素相同，即  $B[i-next[i-1]-1..i-1]$  与  $B[1..next[i-1]]$  相同
    - \* 关注  $B[next[i-1]+1]$ ，由  $next[]$  的性质可知， $B[next[i-1]+1]$  之前的  $next[next[i-1]+1]$  个元素与前  $next[next[i-1]+1]$  个元素相同，即  $B[next[i-1]-next[next[i-1]+1]..next[i-1]]$  与  $B[1..next[next[i-1]+1]]$  相同
    - \* 可以比较  $B[next[next[i-1]+1]+1]$  与  $B[i-1]$   
若匹配则  $next[i] = next[next[i-1]+1] + 1$   
若不匹配则进行  $j = next[j] + 1$  之后再次匹配  
直到匹配成功，此时  $next[i] = j$ ；或  $j = 0$ ，此时  $next[i] = 0$

```
// Algorithm: next_in_KMP
// Input: string key
// Output: int[] next
int next[key.length];
next[1] = -1, next[2] = 0;
for (int i=3 to key.length) begin
    int j = next[i-1]+1;
    while key[i-1] != key[j] && j>0 begin
        j = next[j]+1;
    endwhile
    next[i] = j;
endfor
return next[];
```

- KMP算法的构建：模式串与目标串出现字符不匹配时，将模式串向右滑动  
直到  $B[i-next[i]-1..i-1]$  与  $B[1..next[i]]$  对齐，比较  $B[next[i]+1]$  与  $B[i]$  即可

```

// Algorithm: KMP
// Input: string obj, string key
// Output: int index
int j = 0;
int [] next = next_in_KMP(key);
for (int i=1 to obj.length) begin
    while (j > 0 && key[j+1] != obj[i]) j = next[j+1];
    if (key[j+1] == obj[i]) j++;
    if (j==key.length) return i-j;
endfor
return 0;

```

### 5.6.3 BM算法

- 思想：在利用模式串特性的基础上，再对失配的目标串字符进行利用
- 坏字符规则：模式串与目标串不匹配时，记目标串字符为坏字符  
出现坏字符时，模式串向右滑动，直到目标串的坏字符与模式串的相同字符对齐（若坏字符在模式串未出现，则使模式串首位与坏字符的右邻位对齐）  
可建立二维数组，表示失配时，根据坏字符，模式串应滑动到的位置
- 好后缀规则：模式串与目标串已匹配的后缀，记这部分字符串为好后缀  
出现失配时，模式串向右滑动，直到模式串中再次出现的好后缀与目标串相同字符串对齐（若好后缀在模式串中完全未出现，则直接向右滑动到目标串中好后缀的右邻位；若好后缀的部分后缀子串与模式串的前缀子串重合，则将模式串移动到和前缀子串重合的位置）  
好后缀规则与KMP算法有部分相似之处

## 5.7 中位数问题

### 5.7.1 加权中位数

问题：给定数组  $X[1..n]$ ，以及  $W[1..n]$  表示各元素权值， $\sum_i w_i = 1$  且  $w_i \neq 0$ ，加权中位数  $x_k$  满足  $\sum_{x_i < x_k} w_i < \frac{1}{2}$ ， $\sum_{x_i > x_k} w_i > \frac{1}{2}$ ，求加权数组的加权中位数

- 直接方法：排序后，从小到大遍历元素，统计  $\sum_{x_i < x_k} w_i$ ，返回首次  $\sum_{x_i < x_k} w_i > \frac{1}{2}$  的前个位置，时间复杂度  $\Theta(n \log n)$

- 快排优化：选择主元将集合分为 2 部分，比较前半权值、后半权值和  $\frac{1}{2}$  的大小，若有一部分大于  $\frac{1}{2}$ ，则在相应部分递归求解（需要修正权值及阈值）

### 5.7.2 等长有序数组的中位数

问题：给定增序数组  $A[1..n], B[1..n]$ ，求这  $2n$  个数的中位数

- 直接方法：归并排序的归并，依次出队  $n$  次，时间复杂度  $\Theta(n)$
- 二分优化：比较  $A\left[\left\lceil\frac{n}{2}\right\rceil\right]$  和  $B\left[\left\lceil\frac{n}{2}\right\rceil\right]$ 
  - 若  $A\left[\left\lceil\frac{n}{2}\right\rceil\right] = B\left[\left\lceil\frac{n}{2}\right\rceil\right]$ ，则均是中位数
  - 若  $A\left[\left\lceil\frac{n}{2}\right\rceil\right] < B\left[\left\lceil\frac{n}{2}\right\rceil\right]$ ，则排除  $A\left[\left\lceil\frac{n}{2}\right\rceil + 1..n\right]$  和  $B\left[1..\left\lceil\frac{n}{2}\right\rceil\right]$  后递归求解
  - 若  $A\left[\left\lceil\frac{n}{2}\right\rceil\right] > B\left[\left\lceil\frac{n}{2}\right\rceil\right]$ ，则排除  $A\left[1..\left\lceil\frac{n}{2}\right\rceil\right]$  和  $B\left[\left\lceil\frac{n}{2}\right\rceil + 1..n\right]$  后递归求解

## 5.8 回文串

问题：找出目标串中尽可能长的回文串

- 直接方法：暴力搜索，时间复杂度  $O(n^2)$
- Manacher算法：求奇数长度的回文串时

归纳假设：

- 已找到分别以前  $i - 1$  个元素为中心的最长回文串半径，记为  $d[i]$
- 已找到的回文串中，右边界最靠右的回文串，其右边界记为  $R$ ，则有  $R \geq i - 1$ （因为至少  $obj[i - 1]$  本身是回文串）

则处理  $i$  时：

- $i > R$ ，则暴力搜索求解  $d[i]$ ，并更新  $R$
- $i \leq R$ ，则  $\exists j, obj[i] = obj[j]$ ，且在某一回文串中，关于中心位置对称
  - \*  $i + d[j] < R$ ，表示以  $i$  为中心的最长回文串，其右边界不超过  $R$ ，所以  $d[i] = d[j]$
  - \*  $i + d[j] \geq R$ ，则可以从  $R + 1$  位置开始，检验关于  $i$  对称的位置，因为  $[i..R]$  范围内的字符以匹配
- 处理偶数长度的回文串：如 123321，可将原字符串中相邻的元素之间加入特殊字符如 1@2@3@3@2@1，此时原字符串中偶数长度的回文串，在新字符串中均为以 @ 为中心的奇数长度的回文串
- 算法时间复杂度：由于  $R$  不会回溯，所以复杂度  $\Theta(n)$



## 5.9 序列修正问题

**问题：**记插入 1 个字符或删除 1 个字符或替换 1 个字符为 1 次操作，求  $A[1..n]$  修改为  $B[1..m]$  的最少操作数

- 动态规划：记  $C(i, j)$  为  $A[1..i]$  变为  $B[1..j]$  的最小代价

1. 初始化：  $C(i, 0) = i, C(0, j) = j$

2. 递推：

$$d(i, j) = \begin{cases} 0 & a_i = b_j \\ 1 & a_i \neq b_j \end{cases} \quad C(i, j) = \begin{cases} C(i-1, j) + 1 & (\text{delete}) \\ C(i, j-1) + 1 & (\text{add}) \\ C(i-1, j-1) + d(i, j) & (\text{replace}) \end{cases}$$

3. 求解：取  $C(n, m)$  即为所求解

时间复杂度：表中每个元素的求解均为  $O(1)$ ，所以总复杂度  $O(mn)$

## 5.10 频繁元素问题

### 5.10.1 查找众数

**问题：**若  $x$  在序列  $X[1..n]$  中出现次数超过  $\frac{n}{2}$ ，称其为序列中的众数，求序列的众数，或说明其不存在

- 直接方法：

– 暴力搜索：统计  $A[1..n]$  中每个元素的出现次数

– 中位数查找：用  $O(n)$  找出中位数，再用  $O(n)$  找出中位数出现的次数是否大于  $\frac{n}{2}$

- 候选轮替：删除序列中的 2 个不同的数后，原序列的众数是新序列的众数，所以可以扫描序列更新候选位置

– 若候选位置为空，则当前元素进入候选位置并设次数为 1

– 若候选位置与当前元素相同，则次数 +1

– 若候选位置与当前元素不同，则次数 -1；此时若次数为 0，则清空候选位置

扫描完成后，众数必存活，非众数也可能存活，需要验证存活者是否为众数

时间复杂度  $O(n)$ ，包括最坏情况

### 5.10.2 频繁元素

**问题：**在数据中找出出现最频繁的  $t$  个元素

- 精确算法：为每种数据取值设立计数器，可在数据规模较小时使用，得到精确的结果
- 近似算法：牺牲部分正确性，以换取空间复杂度的较大优化  
借鉴候选轮替思路，只保留  $k$  个计数器，扫描全数组

- 若当前元素有对应计数器，则对应次数 +1
- 若当前元素没有对应计数器
  - \* 若有空计数器，则填入当前元素，且将次数设为 1
  - \* 若没有空计数器，则将所有计数器 -1，并丢弃所有次数为 0 的计数器和当前元素

正确性分析：出现次数  $\geq \frac{n}{k+1}$  的元素一定会出现在计数器中，但计数器中的元素不一定都出现次数  $\geq \frac{n}{k+1}$ ，还需进一步验证出现次数是否  $\geq \frac{n}{k+1}$   
 特别地， $k=1$  时，即为查找众数的候选轮替思路

## 5.11 最长单调序列

**问题：**给定由不同整数组成的序列，求最长递增子列（子列放回原序列时，不必连续，但元素在子列中的相对位置必须与原序列相同）

- 归纳假设1：已经求出原序列前  $m$  位的最长递增序列
- 归纳假设2：已经求出原序列前  $m$  位中末位数最小的递增子列
- 归纳假设3：原序列前  $m$  位中，最长递增子列长度为  $k$ ，已经求出所有长度  $l < k$  的子序列中，末位数最小的递增子列，记为  $BIS(l)$

根据归纳假设， $BIS(l)$  关于  $l$  是递增的（否则更小的数可以添加在更长的序列后面），因此扩充  $BIS$  数组时可以使用二分法查找

理论复杂度  $O(n \log n)$

## 5.12 查找前两大元素

**问题：**给定序列，求序列的最大元素和次大元素

- 直接方法：2 轮遍历，每次分别找出最大元素， $T(n) = 2n - 3$
- 分治方法：
  - 常规分治：只需要比较  $M_1, M_2$ ，若  $M_1 \geq M_2$  则比较  $m_1, M_2$ ，否则比较  $M_1, m_2$
  - 间接分治：对于  $A[i]$  与  $A\left[i + \frac{n}{2}\right]$ ，调整位置使得  $A[i] \leq A\left[i + \frac{n}{2}\right]$   
 在  $A\left[\frac{n}{2} + 1..n\right]$  中找到最大数  $A\left[\frac{n}{2} + f\right]$  即为数组最大数  
 在  $A\left[\frac{n}{2} + 1..n\right]$  中找到次大数  $A\left[\frac{n}{2} + s\right]$ ，则数组的次大数必为  $A[f]$  和  $A\left[\frac{n}{2} + s\right]$  其中之一

$$T(2) = 1, T(n) = \frac{n}{2} + T\left(\frac{n}{2}\right) + 1 \quad T(n) = n + \log n - 2$$

- 候选法：将元素两两比较，并将结果保留在二叉树中，选出最大数后回溯曾与最大数比较过的元素，在其中选出数组的次大数

### 5.13 求多重集合的模

**问题：**定义多重集合为元素可重复的集合，定义多重集合的模为出现频率最高的元素，求给定多重集合的模

- 直接方法：每遇到元素就遍历数组，并更新最高频元素，复杂度  $O(n^2)$
- 基于排序：排序后顺序统计，复杂度  $O(n \log n)$
- 快排优化：将集合分为  $< pivot, pivot, > pivot$  3 部分，前 2 部分递归求解找最高频数的元素，并与  $pivot$  的频数比较，复杂度  $O(n \log n)$
- 递归终止优化：分治前后两部分时，若前后 2 部分的规模已经小于  $pivot$  的频数，则可提前结束分治，复杂度  $O(n \log \frac{n}{M})$

## 6 图算法

### 6.1 图的基本概念 (先修课程知识)

- 机内表示：邻接表
- 默认区分有向图和无向图，不考虑自环和重边
- 无向图只有度的概念，有向图区分入度和出度的概念
- 道路与回路，简单道路与简单回路
- 连通图，连通分支
- 树，森林
- 子图，生成树
- 偶图（二分图）
- 加权图
- 欧拉图与欧拉半图，欧拉回路与欧拉道路
- 哈密顿图与哈密顿半图，哈密顿回路与哈密顿道路

### 6.2 图的遍历

#### 6.2.1 深度优先搜索 (DFS)

DFS的流程为：从任意结点开始，标记其访问过，然后任选与之相邻且未被访问的顶点，递归调用；若找不到与当前结点相邻且未被标记的结点则递归终止，时间复杂度  $\Theta(|V| + |E|)$

- 用途：求连通子图个数；给图的顶点编号；建立DFS生成树
- 无向图的DFS树：非树边都是跨代边（在DFS树中连接具有祖孙关系的结点）
- 有向图的DFS树：非树边包括
  - 后退边：由后代结点指向祖先结点
  - 前向边：由祖先结点指向后代结点
  - 交叉边：关联的两个结点无祖孙关系，但在树的生成过程中，由后访问的结点指向先访问的结点
- 闭链：除回路的首尾结点重合外，不含重复结点和重复边的回路  
在有向图中，有向闭链还要求且边的方向与有向图的方向一致  
有向图DFS后有后退边  $\Leftrightarrow$  有向图含有有向闭链

#### 6.2.2 广度优先搜索 (BFS)

BFS的流程为：从任意结点开始，标记其访问过，然后访问它的所有儿子结点，之后选择任意访问过的儿子结点递归调用；直到遍历全图，时间复杂度  $\Theta(|V| + |E|)$

**BFS树性质:**

- 树中的父结点一定比所有儿子结点更早访问
- 无权图中，根结点到所有结点的最短路径为BFS树中结点的层次
- 非树边连接的结点，层次数至多相差 1

## 6.3 拓扑排序

**问题:** 称不含有向闭链的有向图为有向非循环图，为有向非循环图中的结点按  $1..|V|$  编号，要求若存在边  $\langle v, w \rangle$  则结点  $v$  的标记小于结点  $w$  的标记

- BFS: 必然能找到入度为 0 的结点  $r$  并标记为 1，之后删除该结点及相关联的边，并对新图递归调用，即为拓扑排序的过程，时间复杂度  $\Theta(|V| + |E|)$

## 6.4 最短路径问题

### 6.4.1 无权有向图的单源最短路径

**问题:** 给定无权有向图  $G = (V, E)$ ，满足从  $v \in V$  出发能到达所有结点，求从  $V$  出发到达所有结点的最短路径

- 归纳: 可对该图进行拓扑排序，设已经知道  $1..n-1$  的最短路径长度  $l[1..n-1]$ ，则对  $n$  只要找到所有存在边  $\langle k, n \rangle$  的结点  $k$ ，并取  $\min_k l[k] + 1$  即可
- 拓扑排序中删除结点  $v$  时，设删除  $\langle v, w \rangle$ ，设从源头到  $v$  的路径长为  $l$ ，则可将暂存的源头到结点  $w$  的路径长最小值与  $l + 1$  比较，并保留更小的值，可在与拓扑排序相同阶数的时间内，求解无权有向图的单源最短路径问题

### 6.4.2 正权有向图的单源最短路径

**问题:** 给定加权有向图  $G = (V, E, W)$ ，满足从  $v \in V$  出发能到达所有结点，且所有边的权值均为正值，求从  $V$  出发到达所有结点的最短路径

- Dijkstra算法: 将所有结点的最小距离初始化为  $\infty$   
更新从  $v$  出发经过 1 条边可达的结点的最小距离，并最终确定上述更新的最小距离中，距离的最小值  $l[x]$   
再更新从  $x$  出发经过 1 条边可达的结点的最小距离，并最终确定上述全部更新的最小距离中，距离的最小值  $l[y]$   
直到最终确定全部结点的最短路径  
时间复杂度  $O((|V| + |E|) \log |V|)$

### 6.4.3 任意权有向图的单源最短路径

**问题：**给定加权有向图  $G = (V, E, W)$ ，满足从  $v \in V$  出发能到达所有结点，允许边的权值为正值或负值或 0，求从  $V$  出发到达所有结点的最短路径

- Bellman-Ford算法：将所有结点的最小距离初始化为  $\infty$   
对所有边  $\langle u, w \rangle$  进行松弛操作：若  $dis[u] + W[\langle u, w \rangle] < dis[w]$ ，则更新  $dis[w]$  为  $dis[u] + W[\langle u, w \rangle]$   
更新  $|V| - 1$  次即可得到任意权有向图的单源最短路径  
时间复杂度  $O(|V||E|)$

### 6.4.4 任意图的全部最短路径

**问题：**给定加权任意图  $G = (V, E, W)$ ，求任意结点对之间的最短路径

- 对边归纳：归纳过程中，每次加入新的边时，检查每对结点之间是否有包含新加入边的最短路径，时间复杂度  $O(|E||V|^2)$
- 对顶点归纳：归纳过程中，每次加入新的结点时，求出新结点到其他顶点的最短路径，还要检查其他结点对之间是否存在经过新结点的更短路径，时间复杂度  $O(|V|^3)$
- Floyd-Warshall算法：按路径类型归纳，从 1 到  $n$  标记各顶点，设从  $u$  到  $w$ ，且路径上最大结点恰好为  $k$  的路径为  $u - w - k - path$ ，若  $u$  和  $w$  之间存在边，成为  $u - w - 0 - path$   
归纳过程中，已知  $u - w - k - path$  的最短长度，求  $u - w - m - path$  的最短长度 ( $m > k$ )，则结点  $m$  必须包含在  $u - w - m - path$  中，则  $u - v - m - path$  的最短长度为  $u - m - i - path$  的最短长度与  $m - w - j - path$  最短长度之和 ( $i < m, j < m$ )  
时间复杂度  $O(|V|^3)$ ，但单步操作简单，可实现性强

### 6.4.5 最短路径算法应用：负环问题

**问题：**给定加权有向图  $G = (V, E, W)$ ，求图中是否存在负环，即有向图中的边权和为负值的有向回路

- Bellman-Ford算法：更新第  $n$  次时，若还有更新，则有负环
- Floyd-Warshall算法：若求出  $v - v - i - path$  的最短长度  $< 0$ ，则有负环

## 6.5 传递闭包问题

**问题：**给定有向图  $G = (V, E)$ ，求有向图  $C = (V, F)$ ，满足：若  $G$  中存在  $v \rightarrow w$  的路径，则  $\langle v, w \rangle \in F$

- 归约的应用：将  $G$  扩展为  $G'$ ，满足： $V(G) = V(G')$ ； $G'$  为有向完全图；若  $e \in E(G)$ ，则  $G'$  中  $w(e') = 0$ ，否则  $w(e') = 1$

对  $G'$  求解最短路径问题, 若  $v \rightarrow w$  的最短路径长为 0, 则  $\langle v, w \rangle \in F$ , 否则  $\langle v, w \rangle \notin F$

- 归约<sup>1</sup>: 将问题  $A$  转化为问题  $B$ , 解决问题  $B$  并通过某种方式变为问题  $A$  的解, 从而求解问题  $A$

## 6.6 最小生成树

问题: 给定无向加权连通图  $G = (V, E)$ , 求边权值和最小的生成树  $T$

- 对图边归纳: 原图的边数从  $< m$  到  $m$  时问题可能不一致, 图也可能变得不连通
- 对树边归纳: 找到了  $k$  条边的最小生成树  $T$ , 如何扩展到  $k + 1$  条边的最小生成树  $T'$ 
  - Kruscal算法: 不断加入未遍历到的最短边, 若加入边时形成了回路, 则去掉该边, 否则将该边加入生成树, 直到遍历所有边
  - Prim算法: 从任意结点开始, 加入与已生成的树相关联的最短边, 若形成了回路则去掉该边, 直到遍历所有边

## 6.7 图的分解

### 6.7.1 双连通分支

无向图中, 若边集相关联的结点中, 每对结点之间至少存在  $k$  条不相交 (没有重复结点和重复边) 的路径, 且该边集加入任意边后均不再满足以上条件, 则称该边集为  $k$  连通分支; 若图仅包含 1 个  $k$  连通分支, 则称该图为  $k$  连通图

图不是双连通的  $\Leftrightarrow$  图中存在结点  $v$ , 删除  $v$  后图不再连通, 称结点  $v$  为关节点

- Menger定理:  $G = (V, E)$  无向连通, 若通过删除结点使得  $u, v$  不连通, 则删除结点数至少为  $u, v$  之间不相交路径的最大数目
- Whitney定理: 图是  $k$  连通的  $\Leftrightarrow$  使图不连通至少要删除  $k$  个结点
- 引理:  $e, f$  属于同一双连通分支  $\Leftrightarrow$  存在闭链  $C$  使得  $e, f \in C$

若将连接双连通分支的边也视为双连通分支, 则  $e$  属于仅属于单个双连通分支

问题: 将图划分为极大的双连通分支

- 双连通树: 将图的双连通分支抽象为结点, 若在图中双连通分支之间有边相连, 则在新图中对应的结点之间有边相连, 则新图一定是树, 称为双连通树 (可按照BFS的顺序构造)

根据双连通分支的性质, 在对边归纳的过程中, 若新加入的边在双连通树中形成了回路, 则可将回路中的结点合并为更大的双连通分支

查找回路复杂度  $\Theta(|V|)$ , 边数  $|E|$ , 则时间复杂度  $O(|V||E|)$

- DFS树与双连通分支:

<sup>1</sup>有关归约的问题, 将在第9章和第10章进一步讨论

- 叶结点一定不是关节点
  - 根结点在只有 1 棵子树时等价于叶结点，多于 1 棵子树时一定是关节点
  - 对于内部结点，若找不到非树边将结点  $v$  的子孙连接到  $v$  的祖先，则  $v$  为关节点
  - DFS优化双连通分支划分：
    - **最高祖先**定义为：从  $v$  出发，只经过后代和非树边，到达的DFS层次数最低的结点  $w$ ，称  $w$  为  $v$  的最高祖先（ $v$  的最高祖先也有可能是  $v$  本身）
    - $v$  的最高祖先是  $v$  后代的最高祖先中DFS层次数最低的结点，或  $v$  关联的非树边能到达的DFS层次数最低的结点
    - $v$  是关节点  $\Leftrightarrow v$  存在后代  $w$ ， $w$  的最高祖先不高于  $v$
  - DFS划分双连通分支算法实现（Tarjan算法）：DFS到达结点  $v$  时，对  $v$  未被遍历的儿子递归调用DFS，找到  $High$  值表示最高祖先的层次数（根节点为 0），待所有后代结点的DFS调用完成后，计算  $High(v)$ ，同时判断关节点
- 时间复杂度  $O(|V| + |E|)$

### 6.7.2 强连通分支

有向图中，若点集满足每对结点之间均有路径，且该点集加入任意结点后均不再满足以上条件，称该点集为强连通分支

- 引理：两个结点属于同一双连通分支  $\Leftrightarrow$  图中存在包含这两个结点的有向回路
- 推论：将强连通分支压缩为结点后，图为有向无环图
- 引理：每个顶点属于且仅属于 1 个强连通分支

**问题：**将有向图划分为强连通分支

- DFS划分：进入结点时，操作数  $+1$ ，并记为该结点的进入值  $pre[v]$ ；离开结点时，操作数  $+1$ ，并记为该结点的离开值  $post[v]$ ，则任意两对结点的（进入值，离开值）区间  $(pre[v], post[v]), (pre[u], post[u])$  只有嵌套或分离情况
  - 若  $\langle u, v \rangle$  存在，则根据  $(pre[v], post[v]), (pre[u], post[u])$  的关系，可判断  $\langle u, v \rangle$  的类型
    - $(pre[u], post[u]) \supset (pre[v], post[v]) \Leftrightarrow \langle u, v \rangle$  为前向边
    - $(pre[u], post[u]) \subset (pre[v], post[v]) \Leftrightarrow \langle u, v \rangle$  为后退边
    - $(pre[u], post[u]) \cap (pre[v], post[v]) = \emptyset \Leftrightarrow \langle u, v \rangle$  为交叉边
  - 凝聚图：将强连通分支压缩为结点，若原图中强连通分支之间有边相连，则新图中对应结点之间有边相连，称新图为凝聚图，则凝聚图是有向无环图
- 凝聚图存在入度为 0 的结点，称为源点，对应原图的源点强连通分支；存在出度为 0 的结点，称为汇点，对应原图的汇点双连通分支
- 若以汇点强连通分支中的任意结点为根调用DFS，则DFS遍历该汇点强连通分支的所有结点后结束，不会涉及其他双连通分支
- $post$  的性质：DFS中  $post$  值最大的顶点一定位于某个源点强连通分支



强连通分支之间只存在单向路径，否则可将多个结点对应的点集压缩为更大的强连通分支

DFS从强连通分支  $A$  到达强连通分支  $B$  后，至少会遍历强连通分支  $B$ ，再回到强连通分支  $A$ ，则  $A$  中部分结点的  $post$  值一定大于  $B$  中所有结点的  $post$  值，故有如上性质

- 算法总结：将全图反转，寻找反转图重点源点强连通分支，对应正图中的汇点强连通分支，在正图中遍历该强连通分支后去掉该子图，从而处理更小规模的问题

### 6.7.3 图分解的应用

- 无向图中偶数长度闭链：必然包含在双连通分支中  
边数多于 1，且含有多于 1 个奇数长度闭链的双连通图，必然存在长度为偶数的闭链
- 有向图中奇数长度闭链：必然包含在强连通分支中  
有向图存在奇数长度闭链  $\Leftrightarrow$  以任意结点为根进行DFS，交替标记结点 (*i.e.*  $\langle u, v \rangle$  存在时，若  $u$  的标记为 1，则  $v$  标记 0；若  $u$  的标记为 0，则  $v$  标记 1)，过程中会发现一条边关联的 2 个结点的标记已经相同

## 6.8 匹配问题

- 定义：图边集的子集，满足其中任意边都没有公共顶点
- 完美匹配：匹配的边关联图中的所有结点
- 无匹配：结点在匹配中没有关联的边
- 最大匹配：具有最大边数的匹配
- 极大匹配：匹配加入任意边后，均不满足任意边没有公共顶点

### 6.8.1 稠密图匹配

问题：给定无向图  $G = (V, E)$ ， $|V| = 2n, \forall v \in V, d(v) \geq n$ ，证明必然存在完美匹配，并设计算法求完美匹配

- 证明过程与算法设计：先寻找任意极大匹配，方法为：遍历全图，不断向已有边集加入边，判断是否有重复顶点，可找到边集  $M$ ，设  $|M| = m$ ，若  $m = n$  则为完美匹配  
若  $m < n$ ，则  $\exists v_1, v_2$  不与  $M$  中的边相关联且  $\langle v_1, v_2 \rangle \notin E(G)$ ，由于  $d(v_1) \geq n, d(v_2) \geq n$ ，从  $v_1, v_2$  至少发出  $2n$  条边，且这  $2n$  条边的另一个结点均与  $M$  中的边相关联  
 $\therefore |M| < 2n \therefore \exists u_1, u_2, \langle u_1, v_1 \rangle, \langle u_1, v_2 \rangle, \langle u_2, v_1 \rangle, \langle u_2, v_2 \rangle$  至少存在 3 条边  
对边集  $M$  进行操作：移除  $\langle u_1, u_2 \rangle$ ，加入  $\langle u_1, v_1 \rangle, \langle u_2, v_2 \rangle$  或  $\langle u_1, v_2 \rangle, \langle u_2, v_1 \rangle$ ，则  $M$  仍然满足匹配且  $|M|++$ ，直到  $m = n$  即找到了图的完美匹配

### 6.8.2 偶图匹配

问题：设  $G = (U, V, E)$ ，其中  $U, V$  且  $U \cup V = \emptyset$ ， $E$  中的边满足： $\{u, v\} \in E \Rightarrow u \in U, v \in V$ ，求偶图的最大匹配

- 贪心算法：先匹配度数较小的点，但不能保证最优解
- 交互路径优化：
  - 交互路径：设  $v \in V, u \in U, u, v$  之间有路径，且  $u, v$  均未在  $M$  中被匹配，称该路径为交互路径  
出现交互路径时，可将交互路径中，已匹配的边与未匹配的边互换，实现  $k \rightarrow k + 1$  条边的优化
  - 交互路径查找：设已有边集为  $M$ ， $M$  中的边从  $U$  到  $V$ ， $\neg M$  中的边从  $V$  到  $U$ ，可将无向图转变为有向图，从未匹配的结点开始，通过图搜索算法DFS/BFS，可找到交互路径  
算法的时间复杂度  $O(|V|(|V| + |E|))$
  - 改进：从未匹配的顶点开始BFS，找到未匹配的叶结点，树中从根结点到叶结点的路径即为交互路径，并删除路径中的结点和边，直到找不到交互路径，时间复杂度  $O(\sqrt{|V|}(|V| + |E|))$

### 6.9 非常稠密图的哈密顿回路

问题：无向图  $G = (V, E)$  满足： $\{v, w\} \notin E \Rightarrow d(v) + d(w) \geq n$ ，求  $G$  中的哈密顿回路

- 反向归纳法：设  $G + \{u, v\} = G'$ ，且  $G'$  中找到了哈密顿回路
  - 若  $\{u, v\}$  不在上述哈密顿回路中，则不影响解的情况
  - 若  $\{u, v\}$  在上述哈密顿回路中，根据  $d(u) + d(v) \geq n$ ，从  $u, v$  至少发出  $n$  条边，而  $G$  除去  $u, v$  外只有  $n - 2$  个结点，所以存在哈密顿回路中的边  $(x_i, x_{i+1})$  满足： $(x_i, v), (x_{i+1}, u) \in E(G)$ ，可将原回路修改为  $(u, \dots, x_i, v, \dots, x_{i+1}, u)$ ，依然为哈密顿回路  
时间复杂度  $O(n^2)$

## 7 几何算法

注意：本章出现的算法均为二维平面问题的算法

### 7.1 基本概念

- 几何要素：点  $P(x, y)$ ，直线  $-PQ-$ ，线段  $PQ$ ，路径  $P_1P_2P_3 \dots P_n$ ，封闭路径（多边形） $P_1P_2P_3 \dots P_nP_1$ ，凸多边形满足多边形任意两点之间的连线均在多边形内部，简单多边形满足多边形任意两边除顶点外不相交
- 方法：画草图以获取算法设计思路

### 7.2 点与多边形

问题：给定点  $p$  及多边形，判断点是否在多边形内部

- 归纳：求以  $p$  为端点的射线与多边形的交点数，若为奇数则点在多边形内部

### 7.3 无序点构造多边形

问题：给定无序点集，求任意以该点集中全部点为顶点的多边形

- 事件处理：选取包含所有点的圆，从圆心开始旋转半径扫描点，每次扫描到点时连接，可以确定多边形  
可以选取横坐标最小的点为圆心，无需确定半径，只需求出斜率并排序，就可生成事件点序列
- 特例：存在多个点与参考点共线  
处理：可按照距离由远及近或由近及远连接

### 7.4 凸包问题

问题：定义点集的凸包为包围点集中所有点的最小凸多边形，则易知：凸包多边形的顶点一定属于所给点集，求所给点集的凸包

- 归纳基础：3 个顶点的凸包为三角形，且已求出  $< n$  个顶点的凸包
- 直接归纳：
  - 若新加入的顶点在已有凸包的内侧，什么也不做
  - 若新加入的顶点在已有凸包的外侧，需要修正凸包
  - 问题：时间复杂度  $\Theta(n^2)$ ，且难以操作
- 顺序处理：按照横坐标递增的顺序处理点集  
设  $P_n$  为横坐标最大的点，可求出其余点与该点的连线中与  $x$  轴正向所成角最大、最小的两条连线  $P_nP_a, P_nP_b$ ，则  $P_a, P_b$  一定位于这  $n$  个点的凸包中

将连线  $P_a P_b$  称为支撑线，则支撑线左侧的凸包不变，只需修正支撑线右侧的凸包

时间复杂度  $T(n) = \Theta(n) + T(n-1)$   $T(n) = O(n^2)$

- 礼物包裹算法：从横坐标最大的点出发，每次寻找与参考点连线和  $x$  轴正向夹角最小的点并依次连接，可以求出闭包

时间复杂度  $T(n) = \Theta(n) + T(n-1)$   $T(n) = O(n^2)$  (凸包上每个点的寻找过程均为  $\Theta(n)$ )

- Graham扫描算法：以横坐标最大的点为参考，求出所有点与该点的连线和  $x$  轴正向的夹角，并排序以生成事件点序列

遇到顶点  $p_{k+1}$  时，求出从  $\overrightarrow{p_k p_{k-1}}$  逆时针旋转到  $\overrightarrow{p_k p_{k+1}}$  的角度

– 若  $\leq 180^\circ$ ，则将  $p_{k+1}$  加入凸包

– 若  $> 180^\circ$ ，则将  $p_k$  从凸包中删除，并求出从  $\overrightarrow{p_{k-1} p_{k-2}}$  逆时针旋转到  $\overrightarrow{p_{k-1} p_{k+1}}$  的角度，依此类推，直到  $\leq 180^\circ$

- Graham复杂度分析：排序的时间复杂度  $O(n \log n)$ ，而每个结点最多加入凸包 1 次并从凸包中删除 1 次，所以按事件点序列处理点集的时间复杂度  $O(n)$ ，低于排序的时间复杂度，所以总时间复杂度  $O(n \log n)$

## 7.5 最近点对问题

问题：给定无序点集，求任意两点之间距离的最小值

- 直接方法：求所有两点距离并找出最小值，时间复杂度  $\Theta(n^2)$

- 直接归纳：每次加入新的点时，求出新点与其他点的距离，并更新最小距离

$T(n) = T(n-1) + \Theta(n)$   $T(n) = \Theta(n^2)$  与直接方法相比无明显优化

- 直接分治：将  $n$  个点平均分为 2 个点集，分别求最小值；归并时，要考虑跨点集的距离

$T(n) = 2T(\frac{n}{2}) + O(n^2)$   $T(n) = \Theta(n^2)$  与直接方法相比无明显优化

- 从一维分治提取思路：一维情况下，相邻点之间的距离才可能是最短距离

– 先排序后扫描  $O(n \log n) + O(n) = O(n \log n)$

– 归并：线性时间求出中位数，分为前后 2 部分，分别求出各部分的最小距离，并找出前一半最右点和后一半最左点，求出距离后更新最小值  $T(n) = 2T(\frac{n}{2}) + O(n)$   $T(n) = O(n \log n)$

- 二维分治：假设求出了  $x < l$  和  $x > l$  内部各点之间距离的最小值为  $d$ ，则只需要考虑  $x \in (l-d, l)$  中的点和  $x \in (l, l+d)$  中的点的距离的最小值，便可完成归并

在考虑点  $(x_0, y_0)$  时，只需要考虑  $y \in (y_0-d, y_0+d)$  内的点；根据距离的对称性，还可缩小为只考虑  $y \in (y_0, y_0+d)$  中的点

根据归纳假设， $x \in (l-d, l+d) \wedge y \in (y_0, y_0+d)$  中只可能有  $\leq 6$  个点，否则与归纳假设中  $x < l$  和  $x > l$  内部各点之间距离的最小值为  $d$  互相矛盾

因此可将  $x \in (l-d, l+d)$  中的点按  $y$  坐标增序排序后，每个点只计算与它前面 5 个点之间的距离，并更新最小距离即可

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n \log n) \quad T(n) = O(n \log^2 n)$$

- 分治优化：两个点集合并时，做好  $y$  坐标增序的排序，从而优化为  $T(n) = 2T\left(\frac{n}{2}\right) + O(n)$   $T(n) = O(n \log n)$

## 7.6 线段交点问题

### 7.6.1 水平垂直线段的交点

**问题：**给定  $n$  条水平线段和  $m$  条竖直线段，求这些线段的所有交点，假设水平线段间两两不共线，竖直线段间两两不共线

- 直接方法：逐一比较  $\Theta(mn)$
- 扫描线与事件点：假设有竖直从左向右平移的无限长直线，且扫描过的区域的交点个数已算好，在扫描线移动过程中
  - 若遇到水平线段左端点，则将线段的纵坐标加入考虑范围
  - 若遇到水平线段右交点，则将线段的纵坐标移出考虑范围
  - 若遇到竖直线段，则考虑范围内的所有水平线段
  - 最坏情况下时间复杂度  $O(mn)$ ，此时任意水平线和任意竖直线都有交点
- 扫描线数据结构：可用优先级队列和动态查找表管理候选线集合  
若相同的横坐标处同时有多个事件，优先处理水平线段左端点事件，其次处理竖直线段事件，最后处理水平线段右端点事件  
总时间复杂度  $O((m+n) \log(m+n) + R)$ ， $R$  为求出的总交点个数
- 扫描线算法核心：事件点序列生成（作出不同处理）与候选事件集合（对象特性描述）

### 7.6.2 任意线段交点

**问题：**给定  $n$  条任意线段，求这些线段的所有交点，假设线段间两两不共线，不存在竖直线段，也不存在  $\geq 3$  条直线交于一点

- 扫描线与事件点：假设有竖直从左向右平移的无限长直线，且扫描过的区域的交点个数已算好，根据扫描线移动过程中，扫描线与各线段的交点按纵坐标排序的变化，求出新的交点
  - 若遇到线段左端点：将该线段的左端点有序插入扫描线状态表，并检查该线段与其在扫描线状态表中相邻的线有无交点，若有则将交点加入事件点序列中
  - 若遇到线段右端点：将该线段从扫描线状态表中删除，并检查有无删除前不相邻而删除后相邻的线段，若有则检查对应线段是否存在交点，并将存在的交点加入

事件点序列中

- 若遇到交点：将交点关联的线段的次序交换，并检查有无交换前不相邻而交换后相邻的线段，若有则检查对应线段是否存在交点，并将存在的交点加入事件点序列中

## 7.7 平面图问题

### 7.7.1 基本概念和性质

- 平面图：由结点和直线边构成的图，边之间除结点外不相交，考虑位置关系
  - 点：上述定义中的结点
  - 线：上述定义中的边
  - 面：由线围成的区域，且总是有且仅有 1 个面无穷
- 扩展欧拉公式： $V + F = E + C + 1$ ， $V$  为结点数， $F$  为面数， $E$  为边数， $C$  为连通分支数
- 定理：具有  $V$  个顶点的平面图最多有  $3(V - 2)$  条边和最多  $2(V - 2)$  个面
 

证明：设除了无界的面之外，所有面均为三角形，图只有 1 个连通分支，则每个面最少有 3 条边，且显然每条边由 2 个面共享

$$3F = 2E \quad V - E + F = 2 \quad \text{解得} \quad F = 2(V - 2) \quad E = 3(V - 2)$$
- 三角划分：将多边形划分为多个区域，每个区域均为三角形，且三角形的 3 个顶点均为原多边形的顶点
 

弦：三角划分中位于多边形内部的线
- 引理： $n$  个顶点的简单多边形存在三角划分，满足： $n - 3$  条对角线将多边形划分为  $n - 2$  个三角形
 

证明： $n = 4$  成立，设  $< n$  都成立，则  $= n$  时，存在 1 条多边形内部的对角线将多边形一分为二，可证明如下：

设多边形最左侧顶点为  $v$ ，与  $v$  相邻的两个结点为  $u, w$ ，连接  $u, w$ ，若线段  $uw$  在多边形内，则  $uw$  将多边形一分为二；若不再多边形内，则  $\triangle uvw$  内必然有多边形的顶点，取这些顶点中最左侧的顶点  $v'$ ，连接  $vv'$ ，则  $vv'$  将多边形一分为二

设将  $G$  划分为  $G_1, G_2$ ，则  $V = V_1 + V_2 - 2$

  - 对角线数  $E = E_1 + E_2 + 1 = V_1 - 3 + V_2 - 3 + 1 = V_1 + V_2 - 5 = V - 3$ ，成立
  - 三角形数  $F = F_1 + F_2 = V_1 - 2 + V_2 - 2 = V_1 + V_2 - 4 = V - 2$ ，成立

### 7.7.2 半平面问题

1 条直线将平面分为 2 部分，每部分称为半平面；该部分包含直线称为闭半平面，不包含直线称为开半平面；可表示为  $ax + by \leq c, ax + by \geq c, ax + by < c, ax + by > c$

问题：给定  $n$  个半平面，求半平面的交集

- 归纳：交集为点/凸多边形/凸集，最多包含  $n$  条边
- 分治： $n = 1$  直接返回， $n > 1$  分为两半，递归求交集，难点是如何归并  
扫描线归并：扫描线与凸多边形最多有 2 个交点，因此扫描线最多保存：与 2 个凸集的 4 个交点，及 4 条交边的右顶点，共 8 个状态，因此能在  $O(n)$  内完成归并，算法时间复杂度  $O(n \log n)$

### 7.7.3 美术馆问题

**问题：**求最小的  $n$ ，使得  $n$  个监控在美术馆的特定位置时，多边形美术馆中的任意点都能被至少 1 个监控拍摄到，假设监控能够无死角旋转

- 三角划分的应用：设  $G$  为多边形，将  $G$  的三角划分中的三角形抽象为结点，有公共边的三角形对应的结点间有边，所形成的图记为  $G^*$   
由于三角划分中每条对角线都将多边形一分为二，因此  $G^*$  的每条边都是割边，所以  $G^*$  是树，且树中结点的度数最多为 3  
可证明：三角划分后的多边形可用 3 种颜色进行有效着色  
设三角形个数为  $n$ ，则  $n = 1$  时显然成立；设  $< n$  时都成立，则  $= n$  时，去掉  $G^*$  的所有叶结点，则新图即为  $< n$  的情况，将叶结点加回来时，只需要令  $G$  中添加仅与原图有 1 条公共边的三角形，只需令新结点与原结点颜色不同即可  
只需用上述着色中的任意 1 种颜色即可实现监控全覆盖，而根据抽屉原理，最少的颜色有  $\lfloor \frac{n}{3} \rfloor$  块，即可用  $\lfloor \frac{n}{3} \rfloor$  个监控即可实现全覆盖

### 7.7.4 单调多边形的三角划分

- 单调折线：曲线  $C$  关于直线  $l$  单调指：曲线  $C$  与任意与直线  $l$  垂直的直线至多有 1 个交点，如  $f(x)$  的图像关于  $x$  轴单调
- 单调多边形：多边形  $P$  关于直线  $l$  单调指：多边形  $P$  的边线可被分为 2 条折线，每条折线都关于直线  $l$  单调
- 单调的判定：若考查多边形是否关于  $x$  轴单调，可找出最左侧顶点和最右侧顶点后，从最左侧顶点出发，依次按照上下折线遍历各顶点的横坐标，若得到的序列是增序列，则多边形关于  $x$  轴单调
- 单调多边形划分：每次遇到顶点时，对于未三角化的部分（若有  $> 2$  个未三角化的顶点，则这些顶点一定在同支上），根据是否位于同支和内角是否大于  $180^\circ$  的情况判断如下：
  - 左图： $v_i, v_{i-1}$  在同支上，且内角  $\angle uv_{i-1}v_i > 180^\circ$ ，则将  $v_i$  加入队列
  - 中图： $v_i, v_{i-1}$  在同支上，且内角  $\angle v_{i-2}v_{i-1}v_i < 180^\circ$   
可划分出  $\triangle v_{i-2}v_{i-1}v_i$ ，（如果有）再考虑  $\angle v_{i-3}v_{i-2}v_i$
  - 右图： $v_i, v_{i-1}$  不在同支上，则无论如何都能切掉  $u..v_{i-1}$

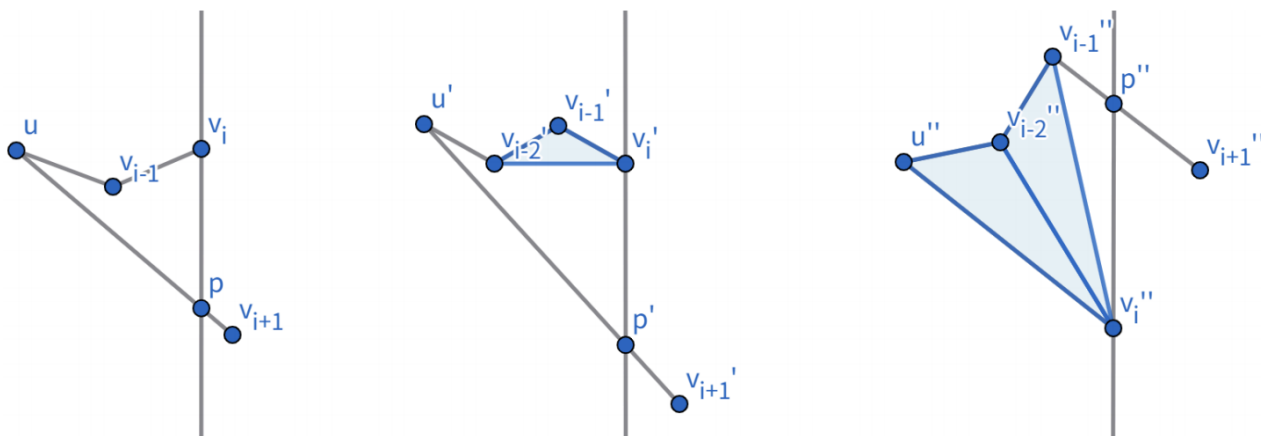


图 1: 单调多边形三角划分示例

### 7.7.5 一般多边形划分为单调多边形

- 转向点：从左向右过程中，若出现横坐标大小变化与前一条边不一致的情况，称为转向点，则从最左点出发遍历上下两支，直到右端点过程中，没有转向点的多边形为单调多边形
  - 若转向点的邻点均在转点右侧，且从该点出发，竖直上下移动，均在多边形内部，称为分离点
  - 若转向点的邻点均在转点左侧，且从该点出发，竖直上下移动，均在多边形内部，称为合并点
- 转向点的消除：按照横坐标增序排序各顶点，设置为事件点序列
  - 分离点  $v$ ：固定扫描线，查找分离点上方的邻边  $e_1$  和下方的邻边  $e_2$ ，查找多边形上  $e_1, e_2$  之间已扫描过的部分中横坐标最大的顶点  $u$ ，连接  $u, v$  即可去除分离点，并将相关多边形内部域分裂为 2 个  
后续遇到其他事件点时，在各自多边形内部域处理即可
  - 合并点：记住该点，扫描线向右移动到下个顶点时，连接该点与合并点即可消除合并点；若与合并点连接的点为分离点，可一并消除
  - 起点：该点的邻点均在该点右侧，且从该点出发，竖直上下移动，均在多边形外部，则创建多边形内部域以处理后续事件点
  - 终点：该点的邻点均在该点左侧，且从该点出发，竖直上下移动，均在多边形外部，则消除多边形内部域并得到单调多边形
  - 上下支的折点：更新多边形内部域内横坐标最大的点，便于分离点的消除
- 正确性证明：设添加了对角线  $uv$ ，且  $x_u < x_v$ ，设直线  $l_1: x = x_u$ ，直线  $l_2: x = x_v$ ， $e, f$  分别为扫描线状态表中， $v$  的上邻边和下邻边，则  $l_1, l_2, e, f$  围成的四边形在多边形内部，否则与  $u$  是多边形上  $e_1, e_2$  之间已扫描过的部分中横坐标最大的顶点矛盾，或与事件点进度表的设置前提矛盾



- 可证明  $uv$  与多边形的边不相交，否则与该四边形在内部相矛盾
- 可证明  $uv$  不与已有对角线相交，因为该四边形中不包含多边形的其他顶点
- 时间复杂度：排序  $O(n \log n)$ ，事件点数量  $\Theta(n)$ ，事件点处理可用优先级队列  $O(\log n)$   
所以从原始多边形到单调多边形时间复杂度  $O(n \log n)$   
又有三角化复杂度  $O(n)$ ，从原始多边形到求出三角划分的总时间复杂度  $O(n \log n)$

### 7.7.6 Voronoi图

- 点的Voronoi区域：设平面内一组基点  $p = \{p_1, p_2, p_3, \dots, p_n\}$ ，则  $p_i$  的Voronoi区域为：满足距  $p_i$  的距离小于到其他所有基点的距离的点的集合，即  $A(p_i) = \{q | \forall j \neq i, |p_i q| < |p_j q|\}$
- Voronoi图：Voronoi区域的边界形成的图
  - Voronoi图的边上的点到 2 个基点的距离相等
  - Voronoi图的顶点到  $\geq 3$  个基点的距离相等
  - 基点的Voronoi区域无界  $\Leftrightarrow$  该基点是基点集凸包上的点
  - Voronoi图只可能是以下之一：
    - \* 若干条平行直线  $\Leftrightarrow$  所有基点共线
    - \* 其边要么是线段，要么是直线
- 定理：Voronoi图的顶点个数最多为  $2n - 5$ ，边数最多为  $3n - 6$ ，其中  $n$  为基点个数  
证明：假设Voronoi图的所有射线视为交于无限远处的 1 个点  
因为Voronoi图也是平面图，根据欧拉公式  $V - E + F = 2$ ，顶点度数之和为  $2E$ ，且每个顶点的度数不小于 3，有  $2E \geq 3V$ ，又  $F = n$ ，代入得  $V \leq 2n - 4$ ， $E \leq 3n - 6$ ，因为  $V$  加上了无穷远处的结点，实际上不存在，所以有如上顶点数和边数性质

问题：生成给定点集的Voronoi图

- 直接法：每个Voronoi区域均为垂直平分线所划分的半平面的交集  
时间复杂度  $O(n^2 \log n)$
- Fortune算法：重点处理已扫描过的区域产生新的事件点问题  
在扫描线移动的过程中，可确定扫描线后方不会发生变化的部分，即到扫描线上方基点的距离小于到扫描线的距离的点构成的区域  
海岸线：扫描线从上往下扫描，以基点为焦点，扫描线为准线的抛物线上方的区域不会再发生变化，将最下方的抛物线段组成的曲线称为海岸线，显然其关于  $x$  轴单调  
断点：海岸线上抛物线的交点称为断点，断点必然在Voronoi边上，所以断点移动的痕迹即为Voronoi图  
扫描线状态表：扫描线的纵坐标，与海岸线相关的交点坐标  
基点事件：每次越过基点时，产生新的抛物线，并拉开原有海岸线，易知抛物线的总段数  $\leq 2n - 1$

顶点事件：由 3 个基点事件动态生成顶点事件，在Voronoi图的顶点处 3 段抛物线合并为 2 段，并在扫描线与 3 个顶点的外接圆相切时生成Voronoi顶点

时间复杂性：  $O(n \log n)$

## 8 代数与数值算法

### 8.1 引言

算法设计考虑因素:

- 复杂性 (算法的时间效率)
- 收敛性 (防止数据溢出)
- 可靠性 (误差在运算过程中会累积, 需要评估误差比例)

误差来源: 模型误差, 观测误差, 截断误差, 舍入误差

### 8.2 求幂运算

问题: 将单步乘法视为  $O(1)$  操作, 设计高效算法求  $n^k$

- 直接归纳:  $n^k = n \cdot n^{k-1}$   
时间复杂度  $O(k) = O(2^{\log k})$  为指数时间算法
- 分治归纳:  $n^k = (k \% 2 == 0) ? (n^{k/2})^2 : n \cdot (n^{k/2})^2$  时间复杂度  $O(\log k)$  为线性时间算法

### 8.3 最大公约数

问题: 求  $n, m$  的最大公约数

- 直接归纳法: 从 2 开始尝试, 直到  $n$  或  $m$ , 时间复杂度  $O(n + m)$
- 辗转相除法: 不妨设  $n > m$ , 设  $GCD(n, m)$  表示  $n, m$  的最大公约数, 则当  $n \% m \neq 0$  时有  $GCD(n, m) = GCD(n \% m, m)$ ,  $n \% m = 0$  时  $n, m$  的最大公约数就是  $m$   
时间复杂度  $O(\log(n + m))$

### 8.4 多项式乘积

问题: 求  $P = p_0 + p_1x + p_2x^2 + \dots + p_nx^n$  与  $Q = q_0 + q_1x + q_2x^2 + \dots + q_nx^n$  的乘积多项式

- 直接法: 时间复杂度  $O(n^2)$
- 分治法:  $P_1 = p_0 + p_1x + p_2x^2 + \dots + p_{\frac{n}{2}-1}x^{\frac{n}{2}-1}$   
 $P_2 = p_{\frac{n}{2}} + p_{\frac{n}{2}+1}x + p_{\frac{n}{2}+2}x^2 + \dots + p_nx^{\frac{n}{2}}$   
 $Q_1 = q_0 + q_1x + q_2x^2 + \dots + q_{\frac{n}{2}-1}x^{\frac{n}{2}-1}$   
 $Q_2 = q_{\frac{n}{2}} + q_{\frac{n}{2}+1}x + q_{\frac{n}{2}+2}x^2 + \dots + q_nx^{\frac{n}{2}}$   
 $PQ = (P_1 + P_2x^{\frac{n}{2}})(Q_1 + Q_2x^{\frac{n}{2}}) = P_1Q_1 + (P_1Q_2 + P_2Q_1)x^{\frac{n}{2}} + P_2Q_2x^n$   
 时间复杂度  $T(n) = 4T\left(\frac{n}{2}\right) + O(n)$   $T(n) = O(n^2)$
- 分治优化: 只需要知道  $P_1Q_2 + P_2Q_1$ , 无需分别求出  $P_1Q_2, P_2Q_1$ , 可直接  $P_1Q_2 + P_2Q_1 = (P_1 + P_2)(Q_1 + Q_2) - P_1P_2 - Q_1Q_2$   
 时间复杂度  $T(n) = 3T\left(\frac{n}{2}\right) + O(n)$   $T(n) = O(n^{\log_2 3}) \approx O(n^{1.59})$

- 应用：在计算大整数乘法时，设每个整数有  $O(n)$  位，若将 1 位加法和 1 位乘法视为  $O(1)$  操作，可用类似的分治优化思路，将时间复杂度从  $O(n^2)$  降为  $O(n^{\log_2 3}) \approx O(n^{1.59})$

## 8.5 方阵乘法

- 定义法：  $C_n = A_n B_n \Leftrightarrow c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$  时间复杂度  $O(n^3)$ ，其中  $n$  为方阵的阶数
- 分治法：

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

$$C = AB = \begin{pmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{pmatrix}$$

时间复杂度  $T(n) = 8T\left(\frac{n}{2}\right) + O(n^2)$   $T(n) = O(n^3)$

- 优化：类似于多项式乘法优化，经过组合后只需要算 7 个乘法  
时间复杂度  $T(n) = 7T\left(\frac{n}{2}\right) + O(n^2)$   $T(n) = O(n^{\log_2 7}) \approx O(n^{2.81})$   
缺点： $n$  非常大时才有优化；输出误差较大；不易并行化

## 8.6 布尔阵乘法

**问题：**矩阵的每个元素都是布尔值，以逻辑与替换乘法，以逻辑或替换加法，注意布尔矩阵运算与数值矩阵运算的不同之处：不能计算减法

- 寻址法：做乘法相加的步骤可视为向量内积  
将向量分段，每一段长为  $k$ ，可在  $O(k \cdot 2^k)$  时间内算出所有可能的  $k$  维向量内积值，之后计算向量内积时，可以按地址查找结果  
 $k = \left\lfloor \frac{1}{2} \log_2 n \right\rfloor$  时，构造表时间为  $O(n \log n)$ ，可在  $O\left(\frac{n}{k}\right) = O(n \log n)$  时间内计算  $k$  维向量的内积，计算  $n^2$  次的复杂度为  $O\left(\frac{n^3}{\log n}\right)$ ，总时间复杂度  $O\left(\frac{n^3}{\log n} + n \log n\right)$   
 $k$  增大时，空间复杂度增加，时间复杂度减小
- 分块法： $AB$  可视为  $A$  第  $i$  列的每个元素与  $B$  第  $i$  行的每个元素相乘，得到矩阵  $M_i$ ，即  $M_i[kl] = A[kl]B[i]$ ，则  $C = \sum_i M_i$   
为加速操作，可一次性取  $A$  的  $k$  列与  $B$  的  $k$  行相乘，具体实现为根据  $A$  第  $i$  行的元素，取出  $B$  的某些行相加，得到结果矩阵的一行  
相加操作可用预计算查表的方式实现，而由于  $A$  的每行均已确定，可能的组合只有  $2^k$  种，而查找到结果后需要以  $\Theta(n)$  时间复制到结果矩阵，所以根据表计算  $M_i$  的时间复杂度为  $\Theta(n^2)$   
构建表的方法是：记  $A$  的  $k$  行可能的组合为  $(0,1)_k$ ，则全为 0 时结果为 0，否则

找到最低位 1 的位置, 求  $xxx \dots x1000 \dots 0$  对应值即为  $xxx \dots x0000 \dots 0$  对应值与  $000 \dots 01000 \dots 0$  对应值之和, 所以表中每行计算时间均为  $\Theta(n)$ , 构建表的复杂度为  $O(n \cdot 2^k)$

总时间复杂度  $\frac{n}{k} [O(n^2) + O(n \cdot 2^k)]$ , 当  $k = \log n$  时复杂度为  $O\left(\frac{n^3}{\log n}\right)$

优化: 布尔加法也可用预计算查表的方式实现, 可将复杂度降为  $O\left(\frac{n^3}{\log^2 n}\right)$

## 8.7 矩阵连乘

问题: 求  $n$  个矩阵 (不一定是方阵) 连乘最少计算次数的计算顺序

- 说明: 根据定义计算  $M_{ij}M_{jk}$  的时间为  $ijk$ ; 对于  $M_{ij}M_{jk}M_{kl}$ , 不同计算方式的计算次数不同

–  $(M_{ij}M_{jk})M_{kl}$  的时间为  $ijk + ikm$

–  $M_{ij}(M_{jk}M_{kl})$  的时间为  $jkm + ijm$

- 暴力遍历法: 令  $M'_1 = \prod_{i=1}^k M_i, M'_2 = \prod_{i=k+1}^n M_i$ , 则  $\prod_{i=1}^n M_i = M'_1 M'_2$ , 计算方法数  $f(n)$  满足

$$f(n) = \sum_{k=1}^{n-1} f(k)f(n-k)$$

$f(n)$  为卡特兰数  $C_{n-1}$ , 因此有  $f(n) = \frac{1}{n} \binom{2n-2}{n-1} = \frac{1}{n} \cdot \frac{(2n-2)!}{((n-1)!)^2} = O\left(\frac{4^n}{n^{1.5}}\right)$ , 计算每种方法的计算次数的复杂度  $\Theta(n)$ , 所以总复杂度  $O\left(\frac{4^n}{\sqrt{n}}\right)$ , 仅比指数时间稍低

- 动态规划法:  $C[i, j] = \min_{1 \leq k \leq j} \{C[i, k-1] + C[k, j] + r_i r_k r_{j+1}\}$ , 问题的解为  $C[1, n]$

复杂度分析: 计算动态规划表每个元素的复杂度  $O(n)$ , 表的规模为  $\Theta(n^2)$ , 总时间复杂度  $O(n^3)$

## 8.8 快速傅里叶变换FFT

- 引入: 多项式的函数曲线及解析式可以用多个点确定

$r(x) = p(x)q(x)$ , 设  $p(x)$  为  $m$  次多项式,  $q(x)$  为  $n$  次多项式, 则选取  $m+n+1$  个不同点  $x_1, x_2, x_3, \dots, x_{m+n+1}$  计算  $p(x_i)q(x_i)$ , 然后解方程组求解  $r(x)$ , 时间复杂度  $O((m+n)^2)$

- 优化: 选择特殊点降低计算量

例如, 选择  $x_i, x_j$  时令  $x_i = -x_j$ , 可减少一半的计算量, 即分别求出奇次项和偶次项的绝对值, 然后根据符号求和, 就能在  $O(n)$  时间内求出  $P(x_i), P(-x_i)$

时间复杂度可优化为  $T(n) = 2T\left(\frac{n}{2}\right) + O(n)$ , 但只对  $n$  成立, 即只能分治 1 次

若要多次递归分治, 则需要引入复数域、四重数域……, 可使上述递推复杂度恒成立, 因此最终的复杂度  $T(n) = O(n \log n)$

解方程组求解  $r(x)$ :

$$\begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \cdots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \cdots & \omega^{(n-1)(n-1)} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{n-1} \end{bmatrix} = \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ \vdots \\ v_{n-1} \end{bmatrix}$$

$$M(\omega)a = v \Rightarrow a = M(\omega)^{-1}v$$

$$\because M(\omega)^{-1} = \frac{1}{n}M\left(\frac{1}{\omega}\right) \quad \therefore a = \frac{1}{n}M\left(\frac{1}{\omega}\right)v$$

## 8.9 (\*)非线性方程组数值解

- 引入: 若将重根视为多个解, 则一元  $n$  次方程在复数域总有  $n$  个解
- 二分法: 逐步缩小区间, 循环结束条件:  $\Delta x < \epsilon$  或  $|f(x)| < \epsilon$
- 迭代法: 令  $g(x) = f(x) + x$ , 则  $f(x) = 0 \Leftrightarrow g(x) = f(x) + x$ , 当  $g(x)$  收敛于其不动点时可用  $x_{n+1} = g(x_n)$  的方式简单迭代求解
- 牛顿切线法:  $x_{k+1} \approx x_k - \frac{f'(x_k)}{f''(x_k)}$   
局部收敛性: 若  $f''(x)$  连续,  $f'(x) \neq 0$ , 则存在根的邻域, 使得牛顿切线法的收敛速度不低于 2 阶, 即

$$\lim_{k \rightarrow \infty} \frac{x_{k+1} - x_0}{(x_k - x_0)^2} = 0$$

## 9 归约

### 9.1 引言

- 定义：将问题  $P$  转化为问题  $Q$ ，分析或求解问题  $Q$ ，并根据问题  $Q$  的解回答问题  $P$
- 用途：根据问题  $Q$  求解问题  $P$ ；通过分析问题  $P$  的复杂度下界，推知问题  $Q$  的复杂度下界

### 9.2 循环平移问题

问题：判断等长的字符串之间是否为循环平移，如‘abcde’和‘cdeab’互为循环平移

- 截断一个字符串为两个字串，然后多次KMP/BM
- 将一个字符串复制一份，拼接到原字符串后面，然后通过一次KMP/BM判断另一个字符串是否为拼接字符串的子串

### 9.3 代表集问题

问题：给定  $k$  个集合  $S_1, S_2, S_3, \dots, S_k$ ，定义代表集  $R = \{r_1, r_2, r_3, \dots, r_k | r_i \in S_i\}$ ，求给定的若干集合是否存在代表集

注意：因为  $r_i \in R, r_j \in R$ ，所以  $r_i \neq r_j$

- 定理： $S_1, S_2, S_3, \dots, S_k$  存在代表集  $\Leftrightarrow \forall I \subset \{1, 2, 3, \dots, k\}, |I| = m, \left| \bigcup_{j=1}^m S_{i_j} \right| \geq m$

直接作为算法的缺点： $\{1, 2, 3, \dots, k\}$  的子集数为  $2^k$ ，算法为指数时间

- 归约：（偶图匹配）设  $G = (V, W, E)$  为二分图，将集合映射到  $V$  中顶点，将元素映射到  $W$  中顶点，求二分图的匹配的过程，即为从每个集合中选出元素组成代表集的过程

### 9.4 序列比较问题

问题与动态规划解法参见第5章5.9节

- 归约：将动态规划表视为有向图，求最短路径问题，则算法的时间复杂度会增大，但可解决问题的类型会增多，比如在不同位置增删改的代价不同时，动态规划较为困难，可归约为最短路径问题解决

### 9.5 无向图的完全子图

问题：寻找  $G = (V, E)$  中是否包含 3 个顶点的完全子图（即三角形）

- 暴力遍历： $\Theta\left(\binom{n}{3}\right) = \Theta(n^3)$

- 归约：设  $A$  为  $G$  的邻接矩阵，则  $A$  为对称布尔阵

根据矩阵乘法定义，有  $A^2[i, j] = \sum_{k=1}^n A[i, k]A[k, j]$

因此若  $A^2[i, j] = 1$ ，则  $\exists k, A[i, k] = A[k, j] = 1$ ，此时只需  $A[i, j] = 1$  则存在三角形  $i, j, k$

布尔矩阵乘法时间复杂度：优化前  $O\left(\frac{n^3}{\log n}\right)$ ，优化后  $O\left(\frac{n^3}{\log^2 n}\right)$ ，均快于暴力遍历

## 9.6 线性规划

线性规划，即将目标问题归约为如下种类的问题：在满足等式约束，不等式约束，变量约束条件下，如何使目标函数取到最值

### 9.6.1 网络流量归约

问题：有向图  $G = (V, E)$  含有入度为 0 的结点  $s$ ，出度为 0 的结点  $t$ ，每条边有权值  $w(e)$  表示  $e$  的最大流量，满足每条边的流量  $f(e) \leq w(e)$ ，除  $s, t$  外每个结点的流入量等于流出量，求网络中流量的最大值（即流入  $s$  或流出  $t$  的流量的最大值）

线性规划：

- 目标函数：  $c(x) = \sum_{i \in S} x_i$ ，其中  $S$  为  $s$  关联的边集
- 约束不等式：  $\forall i, x_i \leq w(x_i)$
- 约束等式：  $\sum_{i \in V_{in}} x_i = \sum_{i \in V_{out}} x_i$
- 变量约束：  $x_i \geq 0$

### 9.6.2 静态路由归约

问题：无向图  $G = (V, E)$  中，每个结点传输信息速率有限（可收可发，结点有缓存），求每条边传输的消息速率，使网络中传输消息数量最多

线性规划：设每条边传输速率  $x_i$

- 目标函数：  $\sum_i x_i$
- 约束不等式：  $\sum_{v_j} x_i \leq \max v_j$
- 变量约束：  $x_i \geq 0$



## 9.7 下界的归约

### 9.7.1 问题的下界

- 问题的下界即问题的复杂度，求解问题的算法复杂度不可能低于问题的复杂度，低于问题复杂度的算法一定是错误的
- 平凡下界：不借助证明，仅借助合理性给出问题的复杂度下界
  - 求全序集的最小值，至少要遍历全集，所以问题下界  $\Omega(n)$
  - 求方阵乘法，至少结果方阵的规模为  $n^2$ ，所以问题下界  $\Omega(n^2)$
- 下界与归约：若问题  $P$  的下界  $\Theta(f(n))$ ，且可根据问题  $Q$  的解改造为问题  $P$  的解，且改造算法的复杂度  $o(f(n))$ ，则问题  $Q$  的下界至少为  $\Theta(f(n))$

### 9.7.2 决策树模型

算法可以看作在一棵树中寻找问题解，所有可能的情况为树的叶结点

设  $L$  是叶结点集， $depth(v)$  表示从根节点到结点  $v$  的路径长，即结点的深度，定义树的外路径总长  $EPL(T) = \sum_{x \in L} depth(x)$ ，则对于二叉决策树有  $EPL(T)_m > |L|(\lceil \log |L| \rceil - 1)$ ，当  $L$  确定时， $EPL(T)$  最小情况是满二叉树（内结点有 2 个子结点，且叶结点集中在最下方 2 层）

根据决策树模型，可分析排序问题的下界：基于比较的排序，决策树叶结点个数  $|L| = n!$ ，平均至少需要  $EPL(T)_{\min}/|L| = \log(n!) - 1$  次比较，所以基于比较的排序算法复杂度均为  $\Omega(n \log n)$

### 9.7.3 简单多边形问题复杂度

简单多边形问题参见第7章7.3节

将排序问题中的点  $\theta_i$  映射为  $(\cos \theta_i, \sin \theta_i)$ ，若可在  $o(n \log n)$  时间内求出简单多边形，则也能在  $o(n \log n)$  时间内排序，与排序问题复杂度  $\Omega(n \log n)$  矛盾，因此简单多边形问题复杂度  $\Omega(n \log n)$

### 9.7.4 矩阵相乘下界归约

- 若可在  $O(T(n))$  时间内计算 2 个  $n \times n$  对称矩阵的乘积，根据定义易知  $T(n) = O(n^3)$ ，所以有  $T(2n) = O(T(n))$ ，则：可在  $O(T(n) + n^2)$  内计算 2 个  $n \times n$  任意矩阵的乘法证明：

$$\begin{pmatrix} O & A \\ A^T & O \end{pmatrix} \begin{pmatrix} O & B^T \\ B & O \end{pmatrix} = \begin{pmatrix} AB & O \\ O & A^T B^T \end{pmatrix}$$

扩展： $O(n^2)$

乘法： $O(T(2n)) = O(T(n))$

提取结果:  $O(n^2)$

总复杂度:  $O(T(n) + n^2)$

- 若可在  $O(T(n))$  时间内计算  $n \times n$  对称矩阵的平方, 根据定义易知  $T(n) = O(n^3)$ , 所以有  $T(2n) = O(T(n))$ , 则: 可在  $O(T(n) + n^2)$  内计算 2 个  $n \times n$  任意矩阵的乘法证明:

$$\begin{pmatrix} O & A \\ B & O \end{pmatrix}^2 = \begin{pmatrix} AB & O \\ O & BA \end{pmatrix}$$

扩展:  $O(n^2)$

乘法:  $O(T(2n)) = O(T(n))$

提取结果:  $O(n^2)$

总复杂度:  $O(T(n) + n^2)$

## 10 P=NP问题

### 10.1 引言

- P-NP性质：
  - P问题：可设计多项式时间的算法求解的问题
  - NP问题：存在多项式时间的非确定算法可求解的问题
  - P和NP针对的是问题本身而非算法
- “计算机不是万能的”悖论：不存在问题“给定算法及输入，判断有限时间内算法能否给出结果”的解决算法，所以计算机不可能解决所有问题

### 10.2 多项式归约 P=NP问题

- 判定问题：答案为肯定或否定问题  
可扩展为最优化问题，且判定问题与对应最优问题的P-NP性质相同
- 多项式归约：设  $U$  为判定问题可能输入的全集， $L \subseteq U$  为使判定问题答案为肯定的输入集合，则称  $L$  为  $U$  的语言  
设  $L_1, L_2$  分别是  $U_1, U_2$  的语言，若存在多项式时间算法将  $u_1 \in U_1$  转化为  $u_2 \in U_2$ ，使得  $u_1 \in L_1 \Leftrightarrow u_2 \in L_2$ ，称  $L_1$  可多项式归约到  $L_2$ ，记为  $L_1 \propto_{poly} L_2$   
多项式归约具有传递性，即  $L_1 \propto_{poly} L_2 \wedge L_2 \propto_{poly} L_3 \Rightarrow L_1 \propto_{poly} L_3$   
但不具有对称性，称  $L_1 \propto_{poly} L_2 \wedge L_2 \propto_{poly} L_1$  时  $L_1, L_2$  多项式等价  
 $L_1 \propto_{poly} L_2$  时， $U_2$  是P问题  $\Rightarrow U_1$  是P问题  
“多项式”针对于输入规模，且  $u_1, u_2$  的规模有多项式关系
- P=NP问题：P问题包含于NP问题是显然的，P=NP问题研究的是P问题的集合是否与NP问题的集合相等，目前该问题的解未知
- NP完全问题：若所有NP问题可多项式归约到  $X$ ，则  $X$  为NP难问题（即不比NP问题复杂度低的问题）；若  $X$  既是NP问题，又是NP难问题，则称  $X$  为NP完全问题

### 10.3 NP完全问题的归约

为方便起见，我们假设合取范式可满足问题的NP完全性已知（NP性证明：随机赋值可在多项式时间验证），并以此来推导其他问题的NP完全性

NP完全性的证明：

- NP性：随机结果可在多项式时间检验
- 归约：证明已知的NP完全问题可归约到待证明的NP完全问题
  - 证明可在多项式时间内转化问题
  - 证明解的可迁移性（等价性）

### 10.3.1 顶点覆盖问题

**问题:**  $G = (V, E)$  为无向图,  $G$  的顶点覆盖是  $V$  的子集, 满足  $E$  中每条边都至少有 1 个顶点在顶点覆盖中, 求  $G$  是否包含  $\leq k$  个顶点的顶点覆盖

- NP性证明: 随机选择  $k$  个顶点, 可在多项式时间内验证每条边是否有顶点在这  $k$  个顶点中
- 归约: (团集问题)

设  $\bar{G} = (V, \bar{E})$  是  $G = (V, E)$  的补图, 即  $(u, v) \in E \Leftrightarrow (u, v) \notin \bar{E}$

可证明  $\bar{G}$  含有  $|V| - k$  个顶点的完全子图  $\Leftrightarrow G$  含有  $k$  个顶点的顶点覆盖, 证明如下:

- 设  $\bar{G}$  中包含  $|V| - k$  个顶点的完全子图, 且其顶点集为  $\bar{U}$ , 则  $(u, v) \in E \Leftrightarrow (u, v) \notin \bar{E} \Rightarrow u \notin \bar{U} \vee v \notin \bar{U} \Leftrightarrow u \in U \vee v \in U$ , 因为根据完全子图的定义,  $u \in \bar{U} \wedge v \in \bar{U} \Rightarrow (u, v) \in \bar{E}$ , 与上述命题互为逆否命题, 二者等价  
 $\therefore (u, v) \in E \Rightarrow u \in U \vee v \in U$ , 即  $U$  是  $G$  的顶点覆盖
- 设  $G$  中包含  $k$  个顶点的顶点覆盖  $U$ , 则  $u \in \bar{U} \wedge v \in \bar{U} \Rightarrow (u, v) \in \bar{E}$ , 因为根据顶点覆盖的定义  $(u, v) \in E \Rightarrow u \in U \vee v \in U$ , 与上述命题互为逆否命题, 二者等价  
 $\therefore \forall u, v \in \bar{U} (u \neq v), (u, v) \in \bar{E}$ , 即  $\bar{U}$  是  $\bar{G}$  的完全子图

又因为图转化为补图为多项式时间, 所以团集问题为NP完全问题  $\Rightarrow$  顶点覆盖问题为NP完全问题

### 10.3.2 支配集问题

**问题:**  $G = (V, E)$  是无向完全图, 设  $D \subseteq V$  满足:  $\forall v \in V, [v \in D] \vee [\exists u \in D, (u, v) \in E]$ , 称  $D$  为  $G$  的支配集, 求  $G$  中是否包含  $\leq k$  个顶点的支配集

- NP性证明: 随机选择  $k$  个顶点, 可在多项式时间内验证每个点及其相邻点是否有顶点在这  $k$  个顶点中
- 归约: (顶点覆盖问题)

设无向完全图  $G = (V, E)$ , 构造新图  $G' = (V \cup V', E')$ , 对  $\forall (v, w) \in E$ , 加入结点  $x \in V'$ , 使得  $(v, x) \in E', (x, w) \in E', (v, w) \in E'$ , 则  $|V \cup V'| = |V| + |E|, |E'| = 3|E|$   
 可证明  $G$  中的顶点覆盖与  $G'$  中的支配集完全等价, 证明如下:

- 设  $G'$  中包含  $k$  个顶点的支配集  $D$ , 检查  $D$  中的顶点是否全部来自  $V$   
 若  $\exists u \in D, u \notin V$ , 则设  $(v, w) \in E', (v, u) \in E', (u, w) \in E'$ , 则用  $v$  或  $w$  替换  $u$ , 而保证  $u, v, w$  都被支配  
 通过此种方式令  $D \subset V$ , 则  $D$  必然包含  $G$  中每条边的至少 1 个顶点, 否则  $V'$  中的顶点不会被支配到  
 $\therefore D$  是  $G$  的顶点覆盖
- 设  $G$  包含  $k$  个顶点的顶点覆盖  $C$   
 由于  $G$  中所有边被  $C$  覆盖, 即  $G$  中每条边至少有 1 个顶点在  $C$  中, 所以

$\forall u \in V' - V$  都至少被  $C$  中的 1 个顶点覆盖  $\therefore C$  是  $G'$  的支配集

又因为图的如上扩充为多项式时间, 所以顶点覆盖问题为NP完全问题  $\Rightarrow$  支配集问题为NP完全问题

### 10.3.3 3-SAT问题

**问题:** 合取范式的每个析取子句都含有恰好 3 个布尔变元, 求该合取范式的可满足性

- NP性证明: 随机给所有布尔变元赋值, 可在多项式时间内判断合取范式是否成立
- 归约: (SAT问题)

设SAT问题的一般形式  $f = x_1 + x_2 + x_3 + \cdots + x_n, F = \sum f$ , 可按照  $f$  中布尔变元的数量, 在多项式时间内将  $f$  重构为  $f'$

–  $k \geq 3$ , 将  $f$  重构为  $f' = (x_1 + x_2 + y_1)(x_3 + \overline{y_1} + y_2) \cdots (x_i + \overline{y_{i-2}} + y_{i-1}) \cdots$

\* 若  $f = 1$  则  $x_1 = 1 \vee x_2 = 1 \vee x_3 = 1 \vee \cdots \vee x_k = 1$

假设  $x_i = 1, \forall j \neq i, x_j = 0$ , 则取  $y_{i-2} = 1, y_{i-1} = 0$ , 则  $x_i + \overline{y_{i-2}} + y_{i-1} = 1$   
依次前递, 令

$$y_k = \begin{cases} 1 & k \leq i-2 \\ 0 & k \geq i-1 \end{cases}$$

则  $f'$  的每个合取子句均可满足, 成立

\* 若  $f' = 1$ , 反证法, 令  $f = 0$ , 则  $x_1 = 0 \wedge x_2 = 0 \wedge x_3 = 0 \wedge \cdots \wedge x_k = 0$ , 根据  $f' = 1$  有

$$y_1 = 1$$

$$\overline{y_1} = 1 \vee y_2 = 1$$

$$\overline{y_2} = 1 \vee y_3 = 1$$

...

$$\overline{y_{i-2}} = 1 \vee y_{i-1} = 1$$

...

$$\overline{y_{k-4}} = 1 \vee y_{k-3} = 1$$

$$\overline{y_{k-3}} = 1$$

依次归结相邻两个子句, 最终得到  $\overline{y_i} = 1 \wedge y_i = 1$ , 矛盾, 因此假设  $f = 0$  不能成立, 所以  $f' = 1 \Rightarrow f = 1$

–  $k = 2, f = x_1 + x_2, f' = (x_1 + x_2 + y)(x_1 + x_2 + \overline{y})$

\* 若  $f' = 1$ , 根据归结证明法  $x_1 + x_2 = 1, \therefore f = 1$

\* 若  $f = 1$ , 则  $f'$  每个析取子句的值均为 1

–  $k = 1, f = x, f' = (x + y_1 + y_2)(x + \overline{y_1} + y_2)(x + y_1 + \overline{y_2})(x + \overline{y_1} + \overline{y_2})$

- \* 若  $f' = 1$ , 根据归结证明法  $x + y_1 + y_2 \wedge x + \overline{y_1} + y_2 \Rightarrow x + y_2$ ,  $x + y_1 + \overline{y_2} \wedge x + \overline{y_1} + \overline{y_2} \Rightarrow x + \overline{y_2}$ ,  $x + y_2 \Rightarrow x + \overline{y_2} \Rightarrow x = 1 \Rightarrow f = 1$
- \* 若  $f = 1$ , 则  $f'$  每个析取子句的值均为 1

所以两式可满足性完全等价, 所以团集问题为NP完全问题  $\Rightarrow$  顶点覆盖问题为NP完全问题

### 10.3.4 团集问题

问题:  $G = (V, E)$  为无向图, 求  $G$  是否包含  $k$  个顶点的完全子图

- NP性证明: 随机选择  $k$  个顶点, 可在  $\Theta(n^2)$  内验证这  $k$  个顶点中是否两两顶点之间有边
- 归约: (合取范式可满足问题) 设SAT问题的一般形式  $C = \sum_{i=1}^m C_i$ , 将  $C$  中出现的所有布尔变量映射到  $G$  中的结点 (重复出现的布尔变元也重复映射), 将可同时取到 1 且不在相同析取子句中的变元, 对应的结点之间用边相连, 即  $E = \{(x, y) | x \neq \overline{y}, (x \in C_i \wedge y \in C_j) \rightarrow i \neq j\}$ , 则问题解可以相互转化, 证明如下:
  - 设SAT问题找到了解, 则每个子句中至少有 1 个变量为 1, 将取 1 的变量对应的顶结点从  $G$  中分离, 由于能同时取到 1 且不在相同析取子句的变元对应的结点之间必然有边, 所以这些结点及其之间的边为完全子图, 于是找到了团集问题的解
  - 设团集问题找到了解, 则将结点对应的变元赋值 1, 可使合取范式满足, 于是找到了SAT问题的可满足解

设SAT问题中的文字数 (单个布尔变元与可能存在的否定符号为 1 个文字, 重复出现的布尔变元也重复计算个数) 为  $n$ , 则可在  $\Theta(n)$  时间内映射到  $V$ , 在  $\Theta(n^2)$  时间内映射到  $E$ , 所以SAT问题是NP完全问题  $\Rightarrow$  团集问题为NP完全问题

### 10.3.5 3着色问题

问题:  $G = (V, E)$  为无向图, 为  $\forall v \in V$  着色, 要求相邻结点颜色不同, 且全图颜色不超过 3 种, 求是否有符合要求的着色方案

- NP性证明: 在 3 种颜色中随机为每个结点着色, 可在  $\Theta(|E|)$  内验证每条边关联的结点的颜色是否不同
- 归约: (3-SAT问题) 初始构建  $\triangle ATF$ , 则  $A, T, F$  的颜色一定各不相同  
 对每个变量  $x$ , 构建  $\triangle Ax\overline{x}$ , 则对于3-SAT问题中一般的析取子句  $x + y + z$ , 可构建出图2, 其中  $X_1, X_2$  分别代表  $x, \overline{x}$ , 依此类推  
 对于3-SAT的每个子句, 再做出图3所示扩展:  
 $I_i$  为内结点, 只与  $I_j (i \neq j), I_k (i \neq k), O_i$  相邻  
 $O_i$  为外结点, 与  $I_i, T$  以及布尔变元对应的结点相邻, 如图4

可证明：若  $G$  能够3着色，则  $x, y, z$  至少有 1 个为  $T$

反证法：首先  $x, y, z$  必不能与  $A$  相同，其次若  $x = y = z = F$ ，则  $O_1, O_2, O_3$  均不能为  $T$  也不能为  $F$ ，只能为  $A$ ，此时内结点  $I_1, I_2, I_3$  均不能为  $A$ ，但  $\triangle I_1 I_2 I_3$  至少需要 3 种颜色才能3着色，矛盾，所以3-SAT问题的解与图的3着色中与  $T$  颜色相同的结点等价

又因为建图过程的复杂度对3-SAT的文字数是多项式阶数的，所以3-SAT问题是NP完全问题  $\Rightarrow$  3着色问题是NP完全问题

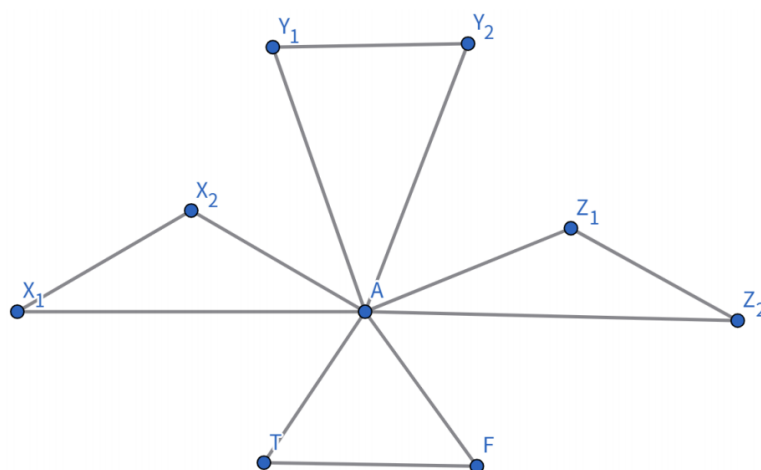


图 2: 析取子句的处理模式

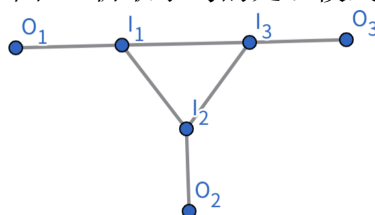


图 3: 辅助部分

### 10.3.6 总结

10.3所列举的NP完全问题之间的归约，如图5所示

## 10.4 (\*) NP完全问题的处理

特例法：如果能直接看出使得判定问题的解答为是的情况，则可直接用特例求解

### 10.4.1 回溯法

- 思想：有组织地穷举，尽量减少穷举的情况（可抽象为DFS）

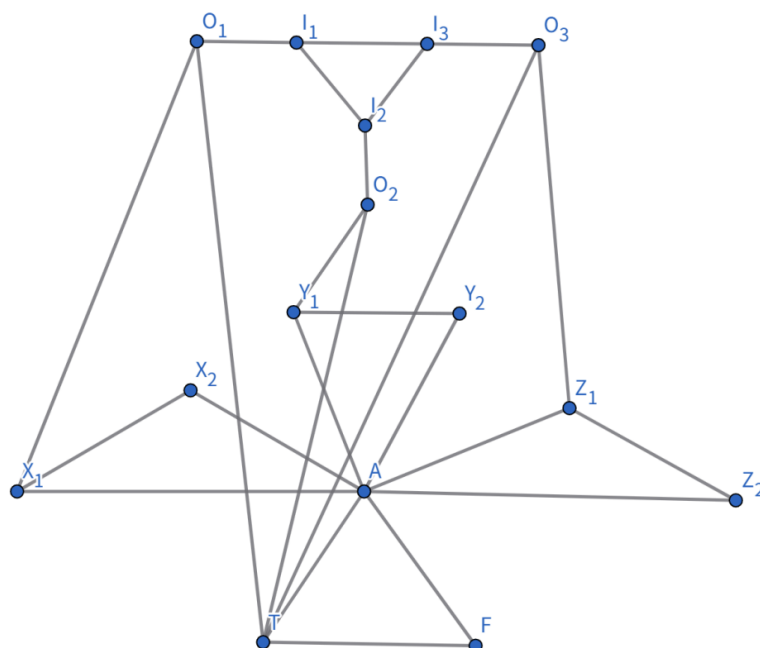


图 4: 组合

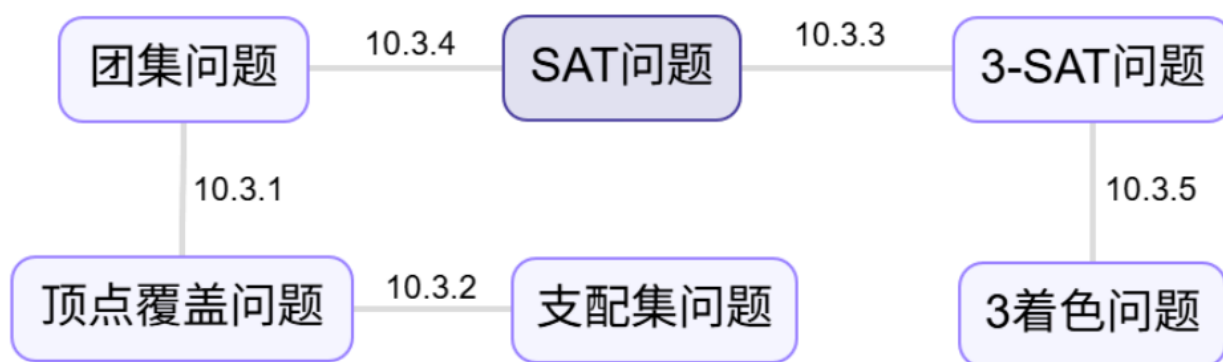


图 5: 总结

- 3着色问题的回溯：从未着色顶点中选一个，避免与相邻已着色结点的颜色相同，多级递归调用，下级结果返回上级循环排除不成立情况

#### 10.4.2 分支限界法

- 思想：回溯法的变形（可抽象为BFS），对最优问题设定上下界，并根据上下界直接排除不合理的分支  
难点：BFS搜索模式、限界函数
- 背包问题的限界：下界为已经放入的物品价值，上界为剩余空间放满当前性价比最高的物品价值，则选择下一个物品时，若所求上界低于当前最优解的上界，则无需考虑
- 旅行商问题的限界：下界为图的邻接矩阵每行最小的 2 个元素相加，最后除以 2，上



界为采用贪心法得到的路径总长，则每次选定下一条边时，若采用贪心法得到的路径总长大于当前最优解的上界，则无需考虑

### 10.4.3 近似算法

- 要求：误差在可控范围内，且时间复杂度有较大优化
- 顶点覆盖的近似：任意极大匹配的所有结点作为顶点覆盖，则算法结果的结点数不超过最小顶点覆盖的 2 倍
- 内存分配的近似：First Fit与Best Fit不会超过最优分配的 2 倍  
改进：先对内存的孔进行排序再进行FF或BF，则误差可以降低到最优解的  $11/9$  倍 +4
- 采用欧氏距离的TSP问题的近似：沿最小生成树行进，结果为最小生成树的 2 倍，不会超过最优解的 2 倍；若沿最小生成树DFS的结点顺序前进，则路径长度可进一步降低，在  $O(n \log n)$  时间内达到了最优解的 2 倍以内的效果  
改进：找出最小生成树中度数为奇数的结点，构建完美匹配并添加到最小生成树，寻找新图中的欧拉回路，则算法结果的路径长不会超过最优解的 1.5 倍

### 10.4.4 随机算法

- 思想：通过概率方法，验证算法的出错率能维持在较低的程度
- 举例：验证 2 个长度为  $n$  的比特串是否相等  
方法：在  $1 \sim n^2$  范围内随机选择素数，传递比特串除以该素数的余数以及素数本身并比较是否相等，则出错率  $\leq \frac{2 \ln n}{n}$   
素数定理： $\leq m$  的素数个数  $\sim \left(\frac{m}{\ln m}\right)^+$   
 $\therefore \leq n^2$  的素数个数  $\sim \left(\frac{n^2}{2 \ln n}\right)^+$  可证明：会导致出错的素数个数  $\leq n - 1$   
设  $p$  会导致错误，则  $(x - y) \% p = 0$ ；设  $\omega = x - y$ ，则  $\omega < 2^n$   
将  $\omega$  素数分解有  $\omega = p_1^{i_1} p_2^{i_2} p_3^{i_3} \cdots p_k^{i_k}$ ，则  $k < n$ ，否则  $\omega$  至少可以分解为  $n$  个  $\geq 2$  的素数相乘，此时  $\omega > 2^n$ ，矛盾，所以会导致出错的素数个数  $\leq n - 1$   
 $\therefore$  出错率  $\sim (n - 1) / \left(\frac{n^2}{2 \ln n}\right)^+ = \left(\frac{n - 1}{n} \cdot \frac{2 \ln n}{n}\right)^- \sim \frac{2 \ln n}{n}$

## 11 (\*) 并行算法

- 复刻加入模型：将主进程分解成多个线程同时进行，可能会有同步问题
- 斐波那契数列的并行：
  - 单线程递归求法：时间复杂度  $O\left(\left(\frac{\sqrt{5}+1}{2}\right)^n\right)$
  - 多线程处理：将任务  $F(n)$  分解为 2 个子任务  $F(n-1)$  和  $F(n-2)$ ，则子任务可以同时处理，但  $F(n)$  需要等待子任务完成后运算 1 步得到
  - 不限制处理器核的数量时，时间复杂度  $O(n)$
- 性能评估：用  $T_p$  表示  $p$  个处理器核下的时间复杂度，有  $T_p \geq \frac{1}{p}T_1, T_p \geq T_\infty$ ，称  $\frac{T_1}{T_p}$  为渐进加速比
- 向量加法的并行：  $y \leftarrow y + Ax \Leftrightarrow (\forall i)y_i \leftarrow y_i + \sum_{j=1}^n A_{ij}x_j$ ，其中  $i$  可以并行， $j$  不可以并行，因为同步问题；可将复杂度由  $\Theta(ij)$  降低为  $\Theta(j)$ ，渐进加速比  $\Theta(i)$
- 定义法计算矩阵乘法的并行：  $c_{ij} = \sum_k a_{ik}b_{kj}$ ， $i, j$  均可以并行化，可将复杂度由  $\Theta(n^3)$  降低为  $\Theta(n)$ ，渐进加速比  $\Theta(n^2)$
- 分治法计算矩阵乘法的并行：

$$AB = \begin{pmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{pmatrix}$$

并行计算 8 个小矩阵乘法、4 个矩阵加法，可将复杂度降低为  $O(\log^2 n)$ ，渐进加速比  $O\left(\frac{n^3}{\log^2 n}\right)$

- 组合优化法计算矩阵乘法的并行：前述优化计算 7 个小矩阵乘法的算法经过并行化后时间复杂度也为  $O(\log^2 n)$ ，渐进加速比  $O\left(\frac{n^{\log_2 7}}{\log_2^2 n}\right)$ ，小于直接分治的加速比，因为本身复杂度低；实际实现并行化时，相比直接分治也更为复杂
- 归并排序并行化：  $T_\infty(n) = T_\infty\left(\frac{n}{2}\right) + O(n) \Rightarrow T_\infty(n) = O(n)$ ，渐进加速比  $O(\log n)$

## A 第6章扩展：机器学习基础

- 机器学习：根据某些系统应有的行为，改善系统的性能
- 决策树模型：根结点为起点，叶结点代表决策，根据样本数据绘制决策树，并根据输入数据和决策树确定输出
- ID3算法：自顶向下构造决策树  
当结点有多个子结点时，将样本数据按照子结点分类  
样本的特征几乎相同时，生成叶子结点  
框架：输入学习样本集，输出决策树
  1. 生成根结点
  2. 若样例都为同类，则直接返回单结点树
  3. 求对样例分类能力最好的属性  $A$
  4. 对样例  $A = v_i (i = 1, 2, 3, \dots, n)$ 
    - 每个分支对应  $A = v_i$  的结果
    - 将样本集按  $A = v_i$  分类
    - 对上述步骤进行递归调用
- 熵模型：定义熵为结果的不确定性  
类比条件概率，可以定义条件熵  
信息增益表示一个条件的确定对另一个条件的熵的减少作用  
条件熵与绝对熵的比值越小，信息增益越大，表示分类能力越强  
缺点：往往会偏向于离散程度更大、取值更多的属性，如时间
- 性能优化的方法：
  - 随机森林：将多棵决策树合成为森林
  - 集成分类：使用多个原理差异很大的分类算法，并将分类算法的结果民主集中为最终结果
  - 分类器生成：每棵决策树生成时只选取部分样本和部分属性
  - 深度随机森林：融合随机森林而成的深度学习成果

## B 第9章扩展：线性规划的求解

- 基本思路：将  $n$  维线性规划映射到  $n$  维空间，每个约束会排除部分空间
- 等势线：使得目标函数取到相同值的子空间，求线性规划的解即为求势的最大值
- 解的情况：
  - 唯一解：势取最大值的情况为一个点
  - 不唯一解：势取最大值的情况为多个点或非一维图形
  - 无解：
    - \* 约束已将全部空间排除，不存在合理空间
    - \* 合理空间无解，等势线可无限绘制，目标函数可取到  $\infty$
- 单纯形法：只求空间顶点的目标函数值
  - 直到找到某个顶点，该处的目标函数值大于其所有相邻顶点
  - 从源点开始出发，放松目标函数中系数为正的约束，使其到达另一个顶点
  - 进行坐标变换，将目前点设为原点，查找变换后目标函数中各参数的系数是否有正值
  - 若有正系数，则继续移动并进行坐标变换，否则退出循环
- 常用算法及复杂度：
  - 单纯形法： $O(c^n)$  常系数较小，适用于小规模情况，实现较为简单
  - 椭球法： $O(n^k)$  用于算法理论分析
  - 内点法： $O(n^k)$  用于软件工程应用